

UNIVERSITY OF PADOVA

DEPARTMENT OF MATHEMATICS

MASTER THESIS IN DATA SCIENCE

ENHANCING INNOVATION DIFFUSION MODELS WITH NEURAL RESIDUAL MODELING FOR IMPROVED FORECASTING

SUPERVISOR

MARIANGELA GUIDOLIN

UNIVERSITY OF PADOVA

CO-SUPERVISOR

MASTER CANDIDATE

NICOLÁS MATÍAS TÁMARA GLÜCK

ACADEMIC YEAR

2024-2026

DEDICATION.

TO MY FAMILY, FOR THEIR LOVE, PATIENCE, AND SUPPORT DURING EVERY STEP OF THIS JOURNEY.

I ALSO WISH TO EXPRESS MY SINCERE GRATITUDE TO MY PROF. GUIDOLIN FOR HER GUIDANCE AND ENCOURAGEMENT THROUGHOUT THIS WORK.

Abstract

Innovation diffusion models such as the Bass model are widely used to describe long-term adoption patterns but often fail to capture short-term fluctuations and volatility. This thesis proposes a hybrid forecasting approach that enhances diffusion models through residual modeling, combining structural diffusion with statistical and neural correction models. Classical and dynamic market potential Bass models are paired with SARIMAX and neural residual models, including Temporal Convolutional Networks and multi-horizon TCNs. The approach is evaluated using battery electric vehicle adoption data for Italy and Norway. Results show that residual modeling consistently improves forecast accuracy, with neural and multi-horizon approaches providing the most stable long-term forecasts. Dynamic market potential improves performance in mature and stable markets, while simpler diffusion baselines remain more robust in volatile settings. Overall, the findings demonstrate that neural residual modeling effectively enhances innovation diffusion forecasts when matched to market conditions and forecasting objectives.

Contents

ABSTRACT	v
LIST OF FIGURES	x
LIST OF TABLES	xiii
1 INTRODUCTION	1
1.1 Background	1
1.2 Problem statement and motivation	2
1.3 Research objectives and research questions	3
1.4 Contributions of the thesis	4
2 LITERATURE REVIEW	5
2.1 Innovation Diffusion and Adoption Processes	5
2.2 The Bass Diffusion Model	6
2.3 Extensions of Bass: Time-Varying / Dynamic Market Potential	8
2.4 Residual Structure in Diffusion Forecasting	10
2.5 Residual Refinement Approaches: From SARIMAX to Neural Residual Models	11
2.5.1 Residual Structure in Nonlinear Diffusion Models	11
2.5.2 ARMAX and SARMAX Refinement	11
2.5.3 Limitations of Linear Residual Refinement	13
2.5.4 Neural Residual Models as Nonlinear Generalization	13
2.5.5 Summary and gap addressed by this thesis	14
3 METHODOLOGY	17
3.1 Data description and preprocessing	17
3.2 Forecasting Setup	19
3.2.1 Train-Test Split	19
3.2.2 Forecast Horizon	19
3.2.3 Evaluation Scales: Raw vs Log and Comparison Principle	19
3.3 Diffusion models	20
3.3.1 Bass diffusion (constant market potential)	21
3.3.2 Dynamic Market Potential diffusion (logistic market potential)	22
3.4 Residual modeling strategies	24

3.4.1	No residual correction (mean-only baseline)	25
3.4.2	SARIMAX residual model	25
3.4.3	LSTM residual model	26
3.4.4	TCN residual model	27
3.4.5	Multi-horizon TCN residual model	28
3.5	Final forecast composition (mean + residual)	28
3.6	Metrics and log vs raw interpretation	29
3.6.1	Mean Absolute Error (MAE)	29
3.6.2	Root Mean Squared Error (RMSE)	29
3.6.3	Raw vs Log Scale Evaluation	29
3.7	Computational cost measurement (training time vs accuracy)	30
4	EXPERIMENTS AND RESULTS	31
4.1	Stationarity diagnostics and motivation for residual models	31
4.2	Italy results (log and raw): comparison across residual refinements	35
4.3	Norway results (log and raw): comparison across residual refinements	37
4.4	Best model selection per country and interpretation	38
4.5	Computational cost vs accuracy trade-off	39
4.6	Forecast Comparison	40
5	DISCUSSION	43
5.1	Residual learning improvements	43
5.2	Why Dynamic Market Potential helped Norway more than Italy	44
5.3	Log vs raw: what is fair to compare and how to decide	45
5.4	Practical interpretation for forecasting and policy/business use	46
5.5	Limitations	46
6	CONCLUSION	49
6.1	Summary of findings	49
6.2	Answer to research questions	50
6.3	Recommendations and future work	50
A	CODE	53
A.1	Bass Diffusion Model	53
A.2	Dynamic Market Potential	57
A.3	SARIMAX	61
A.4	Diffusion + Neural residual models	62
A.5	MH-TCN	68
A.6	Results	73
	REFERENCES	77

Listing of figures

4.1	Monthly BEV registrations in Italy	31
4.2	Monthly BEV registrations in Norway	32
4.3	STL decomposition of BEV registrations	32
4.4	Seasonality analysis of BEV registrations in Italy	33
4.5	Seasonality analysis of BEV registrations in Norway	33
4.6	Autocorrelation diagnostics (ACF/PACF) for BEV registrations	34
4.7	Forecast produced by the Bass model with residual refinement for Italy. The figure compares observed BEV registrations with the predicted trajectory over the forecasting horizon.	40
4.8	Forecast produced by the Dynamic Market Potential model with residual refinement for Norway. The predicted trajectory is compared with observed BEV registrations over the forecasting horizon.	40
4.9	Best Forecast model produced by the Bass model with MH-TCN for Italy. . .	41
4.10	Best Forecast model produced by the Dynamic Market Potential model with MH-TCN for Norway.	41

Listing of tables

4.1	Stationarity tests for BEV registration series	33
4.2	Forecast performance comparison on the raw scale	35
4.3	Forecast performance comparison on the log scale	36
4.4	Forecast performance comparison on the raw scale (Experiment 2)	37
4.5	Forecast performance comparison on the log scale (Experiment 2)	38
4.6	Forecast improvement and computational cost for Bass-based models	39
4.7	Forecast improvement and computational cost for Dynamic Market Potential models	39

1

Introduction

1.1 BACKGROUND

In recent years, the transition to sustainable transportation has accelerated significantly. Battery Electric Vehicles (BEVs) play a central role in decarbonization efforts worldwide. Governments have introduced subsidies, expanded charging infrastructure, and implemented regulatory measures to support their adoption [1] [2]. At the same time, advancements in battery technology and increased consumer awareness have contributed to substantial growth in BEV registrations across many countries.

The adoption of BEVs follows a pattern similar to that observed for most high-technology products. It is a gradual process that unfolds over time as the innovation reaches different segments of the market. After early adopters enter the market, subsequent groups begin to adopt the technology, resulting in the characteristic S-shaped diffusion curve commonly observed in technological adoption processes [3] [4].

Accurate forecasting of BEV adoption has become critical for policymakers. Reliable forecasts support the design of effective subsidy schemes, infrastructure planning, and the assessment of market maturity. For firms, forecasting plays a key role in strategic decision-making, influencing investments, production planning, and long-term business strategies.

To address these forecasting needs, a range of modeling approaches has been developed. In particular, innovation diffusion models have been studied in the literature [5] [6]. Among

these, the Bass model and its extensions are among the most commonly applied frameworks [4]. These models provide a structural representation of adoption dynamics by distinguishing between innovators and imitators and modeling how their interactions drive diffusion. While diffusion models are effective in capturing long-term adoption trends, they often struggle to account for short-term fluctuations observed in real-world data.

In parallel, machine learning methods have advanced rapidly, leading to the development of flexible neural network architectures capable of modeling complex temporal patterns. Although these approaches can achieve high predictive accuracy, they often lack theoretical interpretability. This raises an important question: can structural diffusion models and data-driven methods be combined to improve forecasting performance while preserving interpretability?

1.2 PROBLEM STATEMENT AND MOTIVATION

Traditional innovation diffusion models are designed to capture the structural adoption curve of new technologies. These models typically assume that deviations from the underlying diffusion trend behave as random noise. However, empirical evidence suggests that residuals from diffusion models often exhibit systematic patterns rather than purely stochastic behavior [7] [8]. In particular, residuals frequently display autocorrelation, seasonality, and nonlinear dynamics, indicating that relevant information remains unexplained by the structural component. Ignoring these patterns can lead to suboptimal forecasting performance.

Classical refinement approaches, such as ARMAX (AutoRegressive Moving Average with Exogenous variables) and SARIMAX models, attempt to address these limitations by modeling residual autocorrelation within a linear framework [9] [8]. While these methods can improve forecasting accuracy, their reliance on linear assumptions limits their ability to capture complex nonlinear dynamics, especially in markets subject to structural changes, policy interventions, and external shocks.

Recent advances in machine learning have introduced flexible forecasting models, such as Long Short-Term Memory (LSTM) networks and Temporal Convolutional Networks (TCNs), which are capable of modeling complex nonlinear temporal dependencies [10] [11]. These approaches often achieve high predictive accuracy; however, they typically lack the theoretical interpretability provided by diffusion models.

This trade-off between interpretability and predictive performance motivates the central research question of this thesis. The objective is to investigate whether residual modeling can be combined with structural diffusion models to improve forecasting accuracy while preserving

interpretability. In particular, this study evaluates the extent to which neural residual models enhance classical diffusion forecasts and examines under which conditions the additional modeling complexity is justified.

1.3 RESEARCH OBJECTIVES AND RESEARCH QUESTIONS

The primary objective of this thesis is to enhance innovation diffusion forecasting models by incorporating residual modeling techniques.

More specifically, the study aims to:

1. Evaluate the forecasting performance of classical Bass and Dynamic Market Potential diffusion models.
2. Analyze the structure of residuals generated by diffusion models.
3. Compare classical residual refinement methods (SARIMAX) with neural residual approaches (LSTM and TCN).
4. Assess whether multi-horizon neural residual models improve long-term forecasting performance.
5. Examine the trade-off between predictive accuracy and computational cost.

Based on these objectives, the main research questions are:

- Do diffusion model residuals contain systematic temporal structure?
- Does classical ARMAX/SARIMAX refinement improve diffusion forecasts?
- Do neural residual models provide additional improvements beyond linear refinement?
- Is the performance gain of neural residual models justified given their computational cost?
- Does the effectiveness of residual refinement depend on market characteristics (e.g., Italy vs Norway)?

1.4 CONTRIBUTIONS OF THE THESIS

This dissertation provides a number of contributions to the existing literature on forecasting as well as diffusion of innovations. An empirical assessment will be made in comparison of structural models of diffusion to the mixed-model approach to the diffusion of BEVs.

Secondly, this thesis enhances the existing ARMAX model refinement framework by incorporating the use of neural networks to form a new nonlinear approach for adjustment of residual errors. Thirdly, there are performance measures provided through multi-horizon neural network architectures for the prediction of the diffusion of EVs.

Finally, the empirical evaluation of the computational efficiency has been completed through an evaluation of the model complexity versus the model's ability to improve forecasting outcomes as a means for providing useful real-world decision-making information for selecting models.

This research ultimately provides a conceptual framework for merging structural innovation diffusion theory with current data-based forecasting methods by creating a hybrid model that balances model interpretability and the ability to predict future outcomes to better serve research objectives.

2

Literature Review

2.1 INNOVATION DIFFUSION AND ADOPTION PROCESSES

When there is a new technology, product, or service being offered to the market there will not be immediate adoption of all consumers. There are those consumers who will adopt right away while others will wait and observe how it performs as well as learn from others who have experienced it. This gradual process is referred to as innovation diffusion [3]. Understanding innovation diffusion will help people understand how fast technology will be adopted. It can also provide insights into how to influence the growth of this emerging technology via policies or market incentives.

One of the earliest and best-known authors on this subject was Rogers in his book *Diffusion of Innovations* [3]. Rogers postulated that the process of adoption is not just one that occurs as a result of individual choices but that many of the choices occur within the context of social interaction. Individuals learn about new technologies through various forms of media and types of communication and from individuals who have experienced the new technology. These social factors are a large influence on the adoption process. According to Rogers, there are five categories of people that emerge as the result of how quickly an individual will adopt new ideas: innovators, early adopters, early majority, late majority, and laggards. These various groups generate an identifiable adoption pattern over time.

In practice, the adoption of many long-lasting technologies follows an S-shaped curve. At

the beginning, adoption is slow because the technology is new, unfamiliar, or uncertain. As awareness increases and more people observe others adopting the innovation, confidence grows and adoption accelerates. Eventually, growth slows again as the market becomes saturated and fewer potential adopters remain. This S-shaped pattern has been widely observed in real-world data, including consumer electronics, medical innovations, and transportation technologies [3] [6].

Researchers have developed innovation diffusion models to quantitatively describe the process of innovation adoption [3] [6]. These models differ from traditional time-series approaches in that they rely on structural representations of adoption behavior, rather than solely extrapolating historical data [6]. In particular, diffusion models explicitly capture the underlying forces that drive product adoption. These forces are typically categorized into two groups: external influences (such as advertising and government policy) and internal influences (such as social interaction and peer-to-peer communication) [4] [3]. By incorporating these mechanisms into a formal modeling framework, innovation diffusion models provide not only accurate forecasts, but also results that are more interpretable and grounded in behavioral theory [6].

The diffusion model approach to forecasting is particularly appropriate for newly emerging innovations, such as battery electric vehicles (BEVs). BEV adoption will be affected by a variety of factors beyond simply past sales, including incentives for adopting BEVs, charging infrastructure, improvements in the technology, and social acceptance [1] [2]. Due to the complexity of the BEV adoption process, pure statistical models may not account for the adoption dynamics of BEVs over an extended period. Innovation diffusion models naturally incorporate the dynamics of market saturation, social interaction, and growth, thereby making them ideal for long-term forecasting in innovation-driven markets.

Diffusion models (such as Bass and many other variations) have been applied extensively for analysis of the adoption of new technology. They provide theory-based representation of overall trend in adoption over time. However, most diffusion models focus on long-term behaviour of users, while few models possess short-term variability similar to what is recorded in real-world cases.

2.2 THE BASS DIFFUSION MODEL

The Bass Diffusion Model, which was introduced by Frank Bass in 1969, is one of the most implemented innovation diffusion models for the study of the life cycle of different products and technology. The model is attractive because it is structural and interpretable: each param-

eter has a behavioral meaning. It is currently applied successfully in many topics, including consumer electronics, pharmaceuticals, and transportation technologies [6][4].

The model explains the growth of a new product in the market over certain period of time based on the flow of new adopters, individuals that are influenced by two sources of information: external influences (mass media, advertising) and internal influences (word-of-mouth). The Bass Model has three fundamental parameters that help explain the behavior of potential adopters:

Market potential (m): This parameter represents the maximum number of sales during the life cycle of the product. In the classical Bass formulation, this parameter is constant, implying that there is a fixed number of potential consumers that will purchase the product as soon as its launched.

Innovation (p): This coefficient quantifies the effect by external influences. It reflects the behavior of “innovators”, their personal beliefs and taste influenced by media exposure.

Imitation (q): This coefficient represents internal influences. It captures the behavior of “imitators” who adopt at later stages by observing and copying the behavior of others, normally done through word-of-mouth communication.

Formally, the expected number of new adopters at time t is given by:

$$f(t) = m \left(p + q \frac{Y(t)}{m} \right) \left(1 - \frac{Y(t)}{m} \right) \quad (2.1)$$

where m represents the total market potential and $Y(t)$ the cumulative number of adopters [4].

While in applied cases the parameters can be clear, in real data estimating these parameters can be problematic because of data quality and error issues.

Researchers have found several common problems when working with diffusion data. First, the errors in the data usually don't stay constant over time. Through a product's life cycle, the data behave different as it was used to behave in the existing model, plus the variability tends to change, which breaks the assumption of having constant variance.

Secondly, the usual statistical tests don't always work as expected. If real errors are presented the parameter estimates won't follow the standard distributions, which means the reported t-statistics can be misleading.

Finally, when data is grouped over time (for example, monthly instead of daily) or when the data stays apart from the model for long periods, the results can become biased. This can make the estimates look more accurate than they actually are. Because of this, researchers need to

carefully handle how the data is grouped and pay close attention to how errors change over time when estimating the Bass model.

It is also important to recognize the model's limitations. As Guidolin (2023) stated [7], the classic Bass model assumes that market potential stays fixed over time, which is often unrealistic in real innovation settings. In practice, market size can grow or change, the model does not explicitly consider marketing mix strategies, and it is best suited for products with a limited life cycle.

2.3 EXTENSIONS OF BASS: TIME-VARYING / DYNAMIC MARKET POTENTIAL

A key limitation of the classical Bass diffusion model is the assumption of a fixed market potential m , representing the total number of eventual adopters. While this simplifies the model, it is often unrealistic for emerging technologies. In practice, the size of the potential market evolves over time as awareness increases, infrastructure develops, and external conditions change.

This is particularly relevant for Battery Electric Vehicles (BEVs), where adoption is influenced by factors such as charging infrastructure, technological improvements, government policies, and social acceptance. As a result, the number of potential adopters grows over time, making the assumption of a constant market size restrictive.

To address this limitation, several extensions of the Bass model introduce a time-varying market potential, denoted as $M(t)$. These models allow the market size to evolve over time, providing a more realistic representation of the diffusion process.

One important contribution in this area is the Guseo-Guidolin Model (GGM), proposed by Guseo and Guidolin [12]. The GGM explains adoption as the result of two interacting processes: the diffusion of communication and the diffusion of adoption. In this framework, market potential is not fixed, but grows over time as more individuals become aware of the innovation.

More specifically, the GGM models market potential as emerging from a communication process. This means that the size of the potential market is directly linked to how information spreads among individuals in terms of awareness and information diffusion.

Despite the fact that the Guseo-Guidolin model can be estimated using adoption data, its practical implementation is complex. The model assumes that market potential depends on a communication process which is not directly observed. As a result, both communication

and adoption parameters must be inferred from the same adoption data. With approximately 130 monthly observations, estimating these as separate components would introduce too many parameters, increasing the risk of instability and confounded results. Therefore, this thesis instead specifies market potential directly as a parametric function of time, achieving the same aggregate effect with fewer parameters and reflecting a choice of estimation stability over model richness.

The starting point is the standard logistic function:

$$L(t) = \frac{1}{1 + \exp(-k(t - t_0))} \quad (2.2)$$

which takes values in $(0, 1)$ and produces the S-shaped growth pattern that characterizes most diffusion processes [3] [4] [12]. This shape is appropriate for market potential, since the pool of potential adopters is expected to grow slowly at first, accelerate as awareness spreads, and eventually stabilize as the market approaches saturation.

However, market potential in this context does not start at zero nor saturate at one. It has a non-zero initial level $M_{\min}$, reflecting the consumers already reachable at the beginning of the observation period, and a finite long-run ceiling $M_{\max}$. Rescaling the logistic function to the interval $[M_{\min}, M_{\max}]$ gives the final specification:

$$M(t) = M_{\min} + \frac{M_{\max} - M_{\min}}{1 + \exp(-k(t - t_0))} \quad (2.3)$$

where $M_{\min}$ represents the initial market size, $M_{\max}$ the long-run market potential, k the growth rate of market expansion, and t_0 the midpoint of the expansion process. This formulation serves as a reduced-form representation of the communication-driven market growth described in the GGM [12], capturing its aggregate effect through a smooth parametric curve without requiring the communication process to be estimated separately.

Allowing market potential to evolve over time improves the flexibility of diffusion models and their ability to capture real-world adoption patterns. This is particularly important in the context of BEV adoption, where infrastructure development, policy interventions, and technological progress continuously influence the size of the potential market [13].

2.4 RESIDUAL STRUCTURE IN DIFFUSION FORECASTING

Innovation diffusion models are designed to capture the main trend of adoption over time. In particular, they focus on modeling the nonlinear growth pattern of new technologies. However, these models often overlook other important components of time series data, such as seasonality, short-term fluctuations, and volatility. As a result, even if the overall trend is well captured, there can still be significant variation around it.

Empirical evidence shows that models such as the Bass model can fit cumulative adoption data well, but may struggle to explain short-term movements in higher-frequency data, such as monthly or quarterly observations. This often leads to residuals that are not purely random, but instead show patterns such as autocorrelation [7]. This indicates that part of the information in the data remains unexplained by the diffusion model.

In practice, observed adoption data does not always follow a perfectly smooth S-shaped curve. Instead, it often deviates from the average trend due to factors such as policy changes, seasonal effects, or shifts in consumer behavior. Because of this, models that only focus on the long-term trend tend to leave structured patterns in the residuals, which can reduce forecasting accuracy.

This suggests that diffusion models alone may not be sufficient, especially when accurate short-term forecasts are needed. Ignoring residual structure can lead to biased predictions and a loss of important information.

To address this limitation, researchers have proposed combining diffusion models with additional time series components that model residual behavior. Classical approaches include ARMAX and SARIMAX models, which capture linear dependencies and seasonal patterns [9] [8]. These methods improve short-term forecasting by modeling how residuals depend on past values.

More recently, nonlinear approaches, including machine learning models, have been used to capture more complex residual patterns. By combining diffusion models with residual modeling techniques, it is possible to improve forecasting performance while maintaining the interpretability of the structural model.

Overall, incorporating residual structure into diffusion forecasting provides a more complete representation of adoption dynamics, particularly in markets affected by volatility, policy changes, and evolving conditions.

2.5 RESIDUAL REFINEMENT APPROACHES: FROM SARIMAX TO NEURAL RESIDUAL MODELS

2.5.1 RESIDUAL STRUCTURE IN NONLINEAR DIFFUSION MODELS

Innovation diffusion models, such as the Bass model and its extensions, are typically estimated as nonlinear regression models. In general form, such models can be written as:

$$y(t) = f(\beta, t) + \varepsilon(t) \quad (2.4)$$

Here $y(t)$ refers to the observed adoption over time, $f(\cdot, t)$ is the deterministic diffusion function and $\varepsilon(t)$ is the error term. In cases where diffusion model estimations are done by taking residual $\varepsilon(t)$ assumptions with zero mean, constant variance, and no autocorrelation (white noise), it is considered that the model has captured the systematic structure of the data.

When real applications come to subject, the assumptions made won't explain effectively the entire information. The behavior of the residuals would have to be considered, as an exploratory effect in order to verify if it is important enough to be captured. Guidolin (2023, Chapter 5) justified the previous thoughts by showing that residuals from diffusion models frequently show positive autocorrelation, seasonal patterns or another temporal dependence form. Afterwards, it can be verified by a graphical inspection, an autocorrelation function plot or existing tests such as Durbin–Watson statistic (Durbin and Watson, 1950).

To sum up, this means that diffusion models, despite describing long-term adoption trajectory successfully, can fail to capture short-term fluctuations causing the loss of important information. This could be less beneficial when adopting forecast analysis as the residual behavior suggests that part of the information in the data remains unexplained by the structural diffusion component.

This observation motivates the need for a refinement step.

2.5.2 ARMAX AND SARMAX REFINEMENT

When residual autocorrelation is detected, one classical solution is to apply an ARMAX refinement. Before introducing ARMAX models, it is useful to recall the structure of ARMA models.

An ARMA(h, k) model can be written as [9] [8]:

$$y_t = \phi_1 y_{t-1} + \cdots + \phi_h y_{t-h} - \theta_1 z_{t-1} - \cdots - \theta_k z_{t-k} + z_t \quad (2.5)$$

where z_t is a white noise process.

An ARMAX model extends this structure by including an external covariate:

$$y_t = \lambda x_t + \text{ARMA terms} \quad (2.6)$$

where x_t is an explanatory variable and λ its coefficient.

In the context of diffusion models, Guseo and Guidolin (2009) [12] proposed using the predicted diffusion values $f(\hat{\beta}, t)$ as the plug-in covariate. The model then becomes:

$$y_t = \lambda f(\hat{\beta}, t) + \text{ARMA terms} \quad (2.7)$$

When $\lambda \approx 1$, this indicates that the diffusion model has appropriately captured the structural trend. The ARMA component then models the remaining temporal dependence.

Conceptually, including the diffusion prediction as a covariate is equivalent to applying an ARMA model to the residual series. However, the ARMAX formulation provides an additional diagnostic tool: the estimate of λ allows evaluation of the adequacy of the underlying diffusion specification [7].

When seasonal effects are present, this approach can be extended to SARMAX models, which include seasonal autoregressive and moving-average terms.

The practical refinement procedure typically follows these steps [7]:

1. Estimate a nonlinear diffusion model $f(\beta, t)$ (e.g., Bass or Dynamic Market Potential).
2. Analyze residuals through graphical inspection and autocorrelation analysis.
3. If autocorrelation is detected, apply an ARMAX or SARMAX model including $f(\hat{\beta}, t)$ as a covariate.
4. Evaluate parameter estimates, especially λ .
5. Select model order using information criteria such as AIC (Akaike, 1974).
6. Update forecasts using the refined model.

ARMAX refinement therefore represents a classical and theoretically grounded solution when diffusion residuals exhibit temporal dependence.

2.5.3 LIMITATIONS OF LINEAR RESIDUAL REFINEMENT

Classical refinement methods such as ARMAX and SARIMAX can improve forecasting accuracy by modeling residual autocorrelation and seasonal patterns. However, these approaches rely on linear assumptions, meaning that relationships between past and current values are assumed to be stable and linear over time. In practice, this assumption may be too restrictive.

In real-world technology adoption data, residual dynamics are often more complex. They may exhibit nonlinear behavior, structural breaks caused by policy changes, long-memory effects, or irregular seasonal patterns. Additionally, adoption processes can be influenced by interactions across multiple time series, further increasing complexity.

Because of these characteristics, linear models may fail to fully capture the underlying residual structure. In particular, adoption dynamics in emerging technology markets often respond nonlinearly to factors such as financial incentives, infrastructure availability, and sudden regulatory changes.

These limitations motivate the use of more flexible modeling approaches that can better capture complex residual patterns and improve forecasting performance.

2.5.4 NEURAL RESIDUAL MODELS AS NONLINEAR GENERALIZATION

Recent advances in machine learning have introduced neural network models capable of capturing complex nonlinear patterns in time series data. Unlike classical linear models, neural networks can learn flexible relationships between past observations and future values, making them suitable for modeling residual structures that are not well explained by linear dynamics.

In the context of diffusion forecasting, neural networks can be interpreted as a nonlinear extension of classical residual refinement methods such as ARMAX. Traditional approaches model residuals using linear autoregressive structures [8]:

$$\varepsilon_t = \text{linear ARMA process} \quad (2.8)$$

In contrast, neural residual models replace this linear assumption with a flexible nonlinear function:

$$\varepsilon_t = g_\phi(\varepsilon_{t-1}, \varepsilon_{t-2}, \dots) \quad (2.9)$$

where g_ϕ represents a neural network with parameters ϕ [14].

This approach follows the same logic as classical refinement. The diffusion model captures the long-term adoption trend, typically characterized by an S-shaped curve, while the neural network models short-term deviations from this trend. By combining these two components, it is possible to improve forecasting accuracy while preserving the interpretability of the structural model.

Among neural architectures, models such as Long Short-Term Memory (LSTM) networks [10] and Temporal Convolutional Networks (TCNs) [11] are particularly suitable for time series modeling, as they are able to capture temporal dependencies and complex nonlinear dynamics.

Overall, neural residual models provide a flexible extension of classical refinement approaches, allowing diffusion models to better capture real-world adoption dynamics.

2.5.5 SUMMARY AND GAP ADDRESSED BY THIS THESIS

The current body of work on innovation diffusion models has emphasized strongly a model's structural interpretability. For example, diffusion models, such as Bass and its variants, provide consistent behavioral meanings and can easily represent future adoption dynamics. Yet as noted in Chapter 5 of Guidolin (2023), the use of residual analysis is viewed mostly as a diagnostic process and not necessarily an opportunity to constrain the relationships among variables through structural modeling. While residual autocorrelation is seen as an important factor, it encompasses a significant, but relatively unused source of predictive information.

At the same time, state-of-the-art neural forecasting models have produced superior forecasting results in time-series data analysis, including electric vehicle forecasting applications, but they were developed independently from diffusion theory. As a result, these models can produce high levels of accuracy while compromising both their interpretability and their economic meaning.

Therefore, there exists a methodological disconnect between the conflicting viewpoints on how best to forecast the rate of diffusion of innovative products. On one hand, diffusion models provide a theoretical basis and interpretability but typically ignore residual correlations to the short run. On the other hand, neural models provide great flexibility and good accuracy but are not theoretically founded.

This dissertation fills this methodological divide by:

- Employing Bass and Dynamic Market Potential diffusion models as theoretical bases.

- Explicitly modeling residual correlations through both classical (SARIMAX) and neural network models.
- Assessing whether added complexity in the structure of models is warranted by a corresponding increase in forecasting efficiency and computational expense.

Through a systematic comparison of the effects of different residual modeling strategies across various markets and scales of data, we provide evidence to show how and when a neural method for refining residuals will provide significant improvement in forecasting the diffusion of innovations. In addition to providing more accurate forecasts, this study provides insights into the integration of structural and data-driven approaches within the area of research on the diffusion of innovations.

3

Methodology

In this chapter, the data, preprocessing procedures, model specifications, and evaluation framework used in the analysis are described. The objective is to assess whether incorporating residual modeling improves the accuracy of BEV adoption forecasts across the selected markets. The datasets used in this study are obtained from [15] [16].

3.1 DATA DESCRIPTION AND PREPROCESSING

The analysis is based on monthly data on Battery Electric Vehicle (BEV) registrations for Italy and Norway. These two markets are selected because they represent different stages of market maturity. Norway is one of the most advanced BEV markets globally, characterized by strong policy support, well-developed infrastructure, and a high share of BEV adoption. In contrast, Italy represents a less mature market, with slower adoption dynamics, more limited infrastructure, and weaker policy incentives.

The comparison between these two markets allows the evaluation of forecasting models under different conditions. In particular, it provides insight into whether more complex models improve predictive performance across both mature and developing adoption environments.

The main variable of interest is the number of new BEV registrations per month. Using monthly flow data (new registrations) instead of cumulative totals allows the analysis to capture both long-term diffusion trends and short-term fluctuations in adoption.

The main variable is defined as:

$$y_t = \text{number of new BEV registrations at month } t \quad (3.1)$$

Two data representations are considered: the raw scale and a log-transformed scale. The raw scale corresponds to the observed number of registrations, while the log-transformed scale is defined as:

$$y_t^{\log} = \log(1 + y_t) \quad (3.2)$$

The log transformation is used to stabilize variance and reduce heteroskedasticity in the time series. In diffusion processes characterized by rapid growth, later observations tend to dominate the error due to their larger magnitude. The log scale mitigates this issue by giving relatively more weight to earlier observations, allowing for a more balanced evaluation across different stages of adoption [8].

Model evaluation is therefore conducted on the log-transformed scale to ensure comparability, while forecasts are interpreted on the raw scale to maintain real-world relevance.

Prior to model estimation, the following preprocessing steps are applied:

- Conversion of the time index to monthly frequency.
- Sorting of observations in chronological order.
- Identification and treatment of missing values.
- Conversion of all variables to numeric format.

No additional smoothing or filtering techniques are applied, in order to preserve the natural structure of the data.

An exploratory data analysis (EDA) is conducted prior to model estimation to assess the statistical properties of the series. In particular, stationarity is tested using the Augmented Dickey-Fuller (ADF) and KPSS tests.

The results indicate that the BEV registration series for both Italy and Norway are non-stationary in levels, even after log transformation. This is consistent with the presence of a strong upward trend, which is typical in innovation diffusion processes.

This non-stationarity further supports the use of structural diffusion models, which are specifically designed to capture non-stationary growth patterns, rather than relying solely on stationary time series methods.

3.2 FORECASTING SETUP

This section describes the forecasting design, including the train–test split, forecast horizon, and evaluation framework. The objective is to ensure a transparent and fair comparison across all models.

3.2.1 TRAIN-TEST SPLIT

To evaluate forecasting performance, the time series were divided chronologically into a training period and a forecasting horizon. All models were estimated using observations up to December 2022, which represents the end of the training sample. The remaining observations were reserved for out-of-sample evaluation.

The forecasting horizon was set to 24 months, allowing the models to generate two years of future predictions. This setup reflects a realistic forecasting scenario in which models are trained using historical data and evaluated on their ability to predict unseen future observations.

Because time-series data are ordered in time, the split was performed chronologically rather than randomly to avoid information leakage from future observations.

3.2.2 FORECAST HORIZON

All models are evaluated using a fixed multi-step forecast horizon of 24 months. This horizon allows the assessment of both medium- and long-term forecasting performance.

In the context of innovation diffusion, a 24-month horizon is particularly relevant, as it provides a forward-looking perspective for policy design and infrastructure planning based on expected future demand.

$$H = 24 \text{ months} \tag{3.3}$$

To ensure comparability across models, the same forecast horizon is applied consistently when generating predictions.

3.2.3 EVALUATION SCALES: RAW VS LOG AND COMPARISON PRINCIPLE

Forecast errors are first computed on the original scale of the data, based on the number of new BEV registrations:

$$y_t \tag{3.4}$$

On this scale, forecast errors are expressed in absolute terms (i.e., number of vehicles), making them directly interpretable for policy and business decisions.

A second evaluation is performed on the log-transformed scale:

$$\log(1 + y_t) \tag{3.5}$$

The logarithmic transformation helps stabilize variance and reduces the influence of large values in later periods. In diffusion processes characterized by rapid growth, later observations tend to dominate error metrics due to their larger magnitude. The log scale mitigates this effect by giving relatively more weight to earlier observations.

As a result, evaluation on the log scale provides a more balanced comparison of model performance across all phases of the diffusion process.

To ensure a fair comparison, all models are evaluated under the same conditions:

- All models use the same training period.
- All models forecast over the same horizon.
- All models are evaluated using identical error metrics.
- Both raw and log scales are reported.

This consistent experimental design allows isolating the effect of residual refinement strategies on forecasting performance.

3.3 DIFFUSION MODELS

Long-term patterns of BEV adoption can be modelled using diffusion models, which serve as the structural foundation for examining BEV adoption. The underlying process through which adoption spreads through innovation and imitation can be replicated within diffusion models by defining meaningful parameters which explain how adoption spreads over time [4].

There are two types of diffusion models to consider:

1. A classical Bass Model with a static market potential.

2. A Dynamic Market Potential extension that incorporates time variation in the total potential market for BEVs.

3.3.1 BASS DIFFUSION (CONSTANT MARKET POTENTIAL)

The classical Bass model is specified in discrete-time flow form. Let y_t denote the number of new BEV registrations at time t , and let Y_{t-1} denote the cumulative number of adopters up to time $t - 1$. The expected number of new adopters at time t is given by [4]:

$$\hat{y}_t = \left(p + q \frac{Y_{t-1}}{m} \right) (m - Y_{t-1}) \quad (3.6)$$

where:

- $p > 0$ is the coefficient of innovation,
- $q > 0$ is the coefficient of imitation,
- m is the (constant) market potential.

Cumulative adoption evolves recursively as:

$$Y_t = Y_{t-1} + \hat{y}_t \quad (3.7)$$

with initial condition $Y_0 = 0$.

The model implies an S-shaped adoption pattern. Initially, when few individuals have adopted, the imitation effect is limited and adoption grows slowly. As adoption increases, social interaction leads to acceleration in growth. Finally, as the market approaches saturation, the number of remaining potential adopters decreases and growth slows down.

PARAMETER ESTIMATION

The parameters (p, q, m) are estimated using constrained nonlinear least squares, minimizing the mean squared error between observed and predicted flows. Two estimation strategies are considered, one for each evaluation scale used in the experiments.

(I) LOG-SCALE ESTIMATION

$$\min_{p,q,m} \frac{1}{T} \sum_{t=1}^T (\log(1 + y_t) - \log(1 + \hat{y}_t))^2 \quad (3.8)$$

This approach reduces the influence of large values in later periods, producing a more balanced fit across all stages of the diffusion process.

(II) RAW-SCALE ESTIMATION When evaluating on the raw scale, a weighted mean squared error is used to prevent high-volume months from dominating the estimation:

$$\min_{p,q,m} \frac{1}{T} \sum_{t=1}^T w_t (y_t - \hat{y}_t)^2, \quad w_t = \frac{1}{y_t + \varepsilon} \quad (3.9)$$

where ε is a small constant that gives less weight to months with very high registrations.

CONSTRAINTS The following constraints are imposed to ensure meaningful parameter values:

$$p \in (10^{-6}, 1) \quad (3.10)$$

$$q \in (10^{-6}, 5) \quad (3.11)$$

$$m > \sum_{t=1}^T y_t \quad (3.12)$$

After estimation, in-sample fitted values and out-of-sample forecasts are generated recursively using the estimated parameters.

3.3.2 DYNAMIC MARKET POTENTIAL DIFFUSION (LOGISTIC MARKET POTENTIAL)

Following the theoretical framework introduced in Section 2.3, where dynamic market potential is motivated by models such as the Guseo-Guidolin Model (GGM), this thesis implements a Bass-type diffusion model with time-varying market potential. Rather than using the full GGM, a simpler logistic function is used to describe how market potential grows over time. The reasons for this choice are discussed in Section 2.3. In short, the logistic function is easier

to estimate and still captures the key idea that the potential market expands gradually before reaching a maximum level.

The market potential is defined as:

$$M(t) = M_{\min} + \frac{M_{\max} - M_{\min}}{1 + \exp(-k(t - t_0))} \quad (3.13)$$

where:

- $M_{\min}$ = initial market potential,
- $M_{\max}$ = long-run market potential,
- $k > 0$ = growth rate of market expansion,
- t_0 = midpoint of the expansion process.

Given the time-varying market potential, the diffusion process follows a modified Bass formulation:

$$\hat{y}_t = \left(p + q \frac{Y_{t-1}}{M_{t-1}} \right) (M_t - Y_{t-1}) \quad (3.14)$$

where:

- $\hat{y}_t$ represents the predicted number of new adopters at time t ,
- Y_{t-1} is cumulative adoption up to time $t - 1$,
- p and q are the innovation and imitation parameters, respectively.

Cumulative adoption evolves recursively as:

$$Y_t = Y_{t-1} + \hat{y}_t \quad (3.15)$$

This formulation reflects a dynamic diffusion process in which market potential evolves over time and determines both the adoption rate and the remaining number of potential adopters.

To ensure numerical stability and meaningful parameter estimates, the following constraints are imposed:

$$p \in (10^{-5}, 0.2) \quad (3.16)$$

$$q \in (10^{-5}, 1.5) \quad (3.17)$$

$$M_{\max} > M_{\min} > 0 \quad (3.18)$$

$$k \in (10^{-4}, k_{\max}) \quad (3.19)$$

Additionally, the midpoint parameter t_0 is constrained to lie within the observed sample window. A mild regularization term is included in the objective function to discourage boundary solutions and extreme growth rates, thereby improving the robustness of parameter estimation.

3.4 RESIDUAL MODELING STRATEGIES

Diffusion models capture the long-term structural trend of BEV adoption, but they do not fully account for short-term fluctuations or seasonal patterns.

After estimating the diffusion models, residuals are defined as:

$$\varepsilon_t = y_t - \hat{y}_t \quad (3.20)$$

where y_t represents observed BEV registrations and $\hat{y}_t$ denotes the corresponding model predictions.

In a purely stochastic setting, residuals would behave as white noise. However, as will be shown in Chapter 4, the residuals exhibit systematic patterns, including:

- Autocorrelation,
- Seasonal patterns,
- Nonlinear short-term dynamics.

To address this, different residual refinement strategies are implemented. The objective is to improve forecasting performance by combining the structural diffusion component with a model of residual dynamics.

The final forecast is therefore defined as:

$$\text{Final forecast} = \text{Diffusion component} + \text{Residual component} \quad (3.21)$$

3.4.1 NO RESIDUAL CORRECTION (MEAN-ONLY BASELINE)

For this case the diffusion mean captures all the adoption and residuals are taken as random noise:

$$\hat{y}_t^{final} = \hat{y}_t^{diffusion} \quad (3.22)$$

This mean-only model serves as a baseline against which all residual refinement strategies are evaluated.

3.4.2 SARIMAX RESIDUAL MODEL

To improve forecasting accuracy, the first residual refinement approach consists of modeling the residual series using a Seasonal ARIMA (SARIMAX) specification [9] [8].

Specifically, the residual process is modeled as a SARIMAX(1, 0, 1) $\times$ (1, 0, 0)₁₂ without a deterministic trend:

$$(1 - \phi_1 L)(1 - \Phi_1 L^{12})e_t = (1 + \theta_1 L)u_t, \quad u_t \sim i.i.d.(0, \sigma^2) \quad (3.23)$$

where L denotes the lag operator, ϕ_1 is the non-seasonal autoregressive parameter, θ_1 is the moving average parameter, and Φ_1 represents the seasonal autoregressive coefficient at lag 12, capturing annual seasonality in monthly data.

The absence of differencing ($d = D = 0$) implies that the residual process is assumed to be stationary. No intercept is included, as residuals are expected to fluctuate around zero.

This specification introduces dependence on lags 1, 12, and 13, allowing the model to capture both short-term persistence and seasonal patterns that are not explained by the diffusion model.

After estimating the model on the training residuals, H -step-ahead forecasts of the residual component, denoted $\hat{e}_{t+h}$, are generated.

The final hybrid forecast is constructed as:

$$\hat{y}_{t+h}^{hyb} = \hat{y}_{t+h}^{base} + \hat{e}_{t+h}, \quad h = 1, \dots, H \quad (3.24)$$

3.4.3 LSTM RESIDUAL MODEL

To capture nonlinear temporal patterns in the residual series, a Long Short-Term Memory (LSTM) neural network is employed. LSTMs are a class of recurrent neural networks designed to model sequential data and capture long-term dependencies [10].

INPUT REPRESENTATION

The model is trained using rolling windows of past residuals. For each time step t , the input consists of:

1. A sequence of past residuals:
$$(\varepsilon_{t-24}, \dots, \varepsilon_{t-1}) \quad (3.25)$$
2. A representation of the calendar month.

A look-back window of 24 months is used, allowing the model to capture both short-term dynamics and yearly seasonal patterns.

The calendar information is incorporated through a learned embedding of the month index, enabling the model to represent seasonal effects in a flexible way.

NETWORK ARCHITECTURE

The network consists of three main components:

1. An LSTM layer that processes the sequence of past residuals and encodes temporal dependencies into a hidden representation.
2. A calendar embedding layer that transforms the month index into a low-dimensional vector.
3. A feedforward layer that combines the LSTM output and the calendar embedding to produce the predicted residual at time t .

Formally, the model learns a nonlinear function of the form:

$$\hat{\varepsilon}_t = g_\phi(\varepsilon_{t-24}, \dots, \varepsilon_{t-1}, \text{month}_t) \quad (3.26)$$

where g_ϕ denotes the neural network parameterized by ϕ .

This approach allows the model to capture nonlinear temporal dependencies and seasonal patterns in the residual series that are not explained by the diffusion model.

3.4.4 TCN RESIDUAL MODEL

A Temporal Convolutional Network (TCN) can be used to model residual dynamics as an alternative to recurrent neural networks. TCNs are convolutional-based architectures that use causal convolutions and dilated convolutions to capture long-range temporal dependencies and model sequential data [11].

INPUT REPRESENTATION

The TCN model accepts a time series of past residual observations as input. The time series will consist of 36 months prior residual observations to allow for enough historical data to be modeled by establishing longer patterns in time series data.

In addition to using the residual input, the model has three auxiliary features:

- lagged residual value (12 months)
- sine transformation of the month index (1 – 12)
- cosine transformation of the month index (1 – 12)

The cyclical transformations of the sine and cosine features can be used by the model to represent seasonality in such a way that they remain continuous.

NETWORK ARCHITECTURE

The TCN processes the input sequence using stacked convolutional layers with increasing dilation factors. This structure enables the model to capture both short-term and long-term dependencies without relying on recurrent connections.

Formally, the model learns a nonlinear mapping of the form:

$$\hat{\varepsilon}_t = g_\phi(\varepsilon_{t-36}, \dots, \varepsilon_{t-1}, \varepsilon_{t-12}, \text{seasonal features}) \quad (3.27)$$

where g_ϕ denotes the TCN parameterized by ϕ .

This architecture allows the model to efficiently capture complex temporal dynamics and seasonal patterns in the residual series that are not explained by the diffusion model.

3.4.5 MULTI-HORIZON TCN RESIDUAL MODEL

While the standard TCN model predicts one residual value at a time, a multi-horizon forecasting approach can improve stability for longer forecasting horizons.

Instead of predicting residuals sequentially, the multi-horizon Temporal Convolutional Network directly predicts several future residuals simultaneously.

Instead of producing a single output, the network predicts a vector of future residuals:

$$(\hat{\varepsilon}_{t+1}, \hat{\varepsilon}_{t+2}, \dots, \hat{\varepsilon}_{t+H}) \quad (3.28)$$

where H represents the forecasting horizon.

3.5 FINAL FORECAST COMPOSITION (MEAN + RESIDUAL)

The forecasting framework proposed in this thesis combines a structural diffusion model with a residual correction model.

Formally, the final forecast can be written as:

$$\hat{y}_t = \hat{\mu}_t + \hat{\varepsilon}_t \quad (3.29)$$

where

- $\hat{\mu}_t$ represents the forecast generated by the diffusion model
- $\hat{\varepsilon}_t$ represents the predicted residual component

The procedure therefore follows three main steps:

1. Estimate the diffusion model parameters using historical BEV adoption data.
2. Compute residuals as the difference between observed values and diffusion model predictions.
3. Train a residual model (SARIMAX, LSTM, or TCN) to forecast the remaining unexplained variation.

3.6 METRICS AND LOG VS RAW INTERPRETATION

The performance of forecasting is measured by using standard error metrics that are calculated in the original or log scale. This provides an overall picture of model accuracy over all phases of the diffusion process.

3.6.1 MEAN ABSOLUTE ERROR (MAE)

$$MAE = \frac{1}{n} \sum_{t=1}^n |y_t - \hat{y}_t| \quad (3.30)$$

The Mean Absolute Error (MAE) provides an estimate of the average magnitude (size) of forecasting error in absolute terms.

3.6.2 ROOT MEAN SQUARED ERROR (RMSE)

$$RMSE = \sqrt{\frac{1}{n} \sum_{t=1}^n (y_t - \hat{y}_t)^2} \quad (3.31)$$

Because RMSE has the squared component, it imposes a greater penalty for larger errors. Plus RMSE is sensitive to outliers. Therefore, if avoiding large forecasting errors is important, RMSE is a useful measure when evaluating model quality.

3.6.3 RAW VS LOG SCALE EVALUATION

Evaluation is conducted on both the raw scale and the log-transformed scale:

$$\log(1 + y_t) \quad (3.32)$$

The raw scale reflects absolute forecast accuracy and is directly interpretable in terms of the number of BEVs. In contrast, the log-transformed scale reduces the influence of large values in later periods and provides a more balanced assessment across different stages of the diffusion process.

In particular, the log scale ensures that early-stage adoption periods are not dominated by high-volume observations in later stages, allowing for a fair comparison of model performance over the entire time series.

3.7 COMPUTATIONAL COST MEASUREMENT (TRAINING TIME VS ACCURACY)

In addition to forecasting accuracy, the computational cost of each modeling approach is evaluated.

The comparison includes:

- diffusion models without residual correction
- diffusion models refined using SARIMAX residuals
- diffusion models refined using neural residual models (LSTM and TCN)

The objective of this analysis is to evaluate whether the improvement in forecasting accuracy justifies the additional computational complexity introduced by neural models.

4

Experiments and Results

This chapter evaluates the performance of innovation diffusion models and assesses whether residual modeling improves forecasting accuracy. The analysis includes an examination of data characteristics, model results for both countries under different residual refinement strategies, and a comparison of model performance. The chapter concludes with an evaluation of whether the use of neural models is justified given their computational cost.

4.1 STATIONARITY DIAGNOSTICS AND MOTIVATION FOR RESIDUAL MODELS

Before conducting formal statistical tests, it is useful to visually inspect the data to understand its main characteristics.

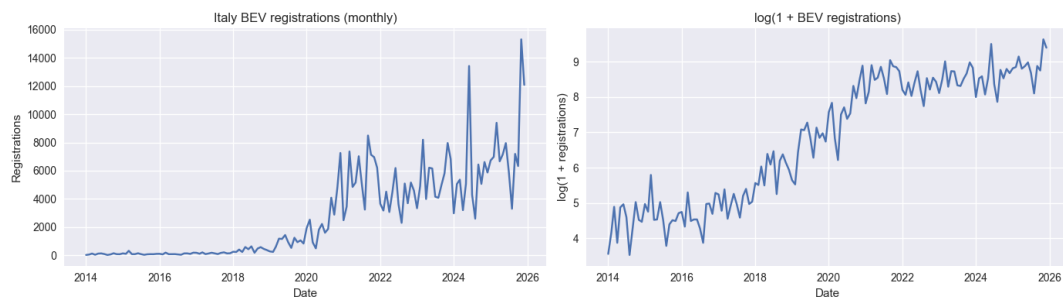


Figure 4.1: Monthly BEV registrations in Italy

Figure 4.1 shows the evolution of BEV registrations from 2014 to 2025. Adoption starts at low levels and increases rapidly over time, with growing variability. The overall pattern follows an S-shaped trajectory, supporting the use of diffusion models such as the Bass model to capture long-term adoption dynamics.

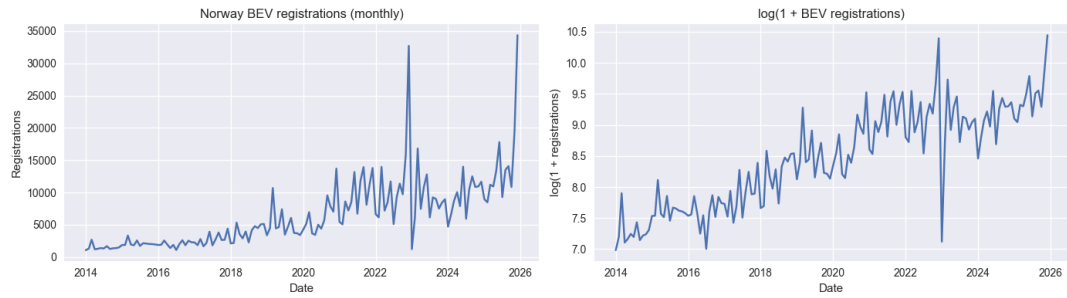


Figure 4.2: Monthly BEV registrations in Norway

Figure 4.2 shows a more stable and mature market. Although the series includes some fluctuations, overall volatility is lower compared to Italy. The adoption pattern suggests a later stage of diffusion, where growth is driven by social interaction and policy support. While the S-shape is less pronounced, the behavior remains consistent with diffusion processes.

To further analyze the data, an STL decomposition and seasonality analysis are performed.

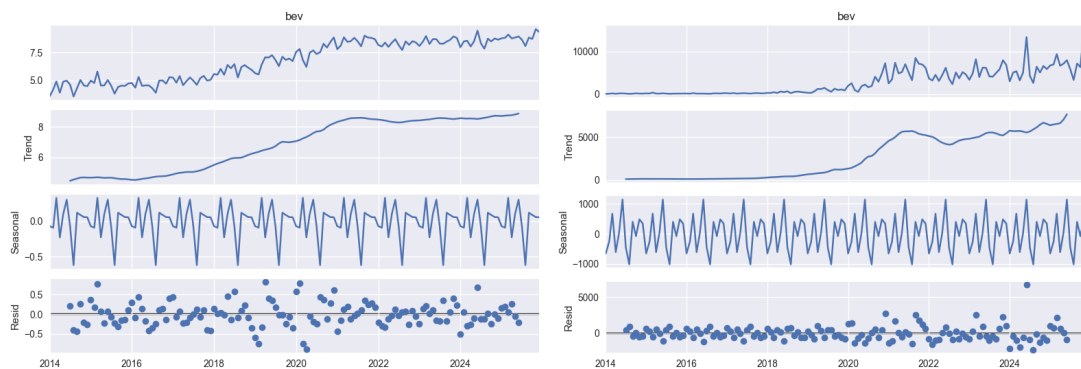


Figure 4.3: STL decomposition of BEV registrations

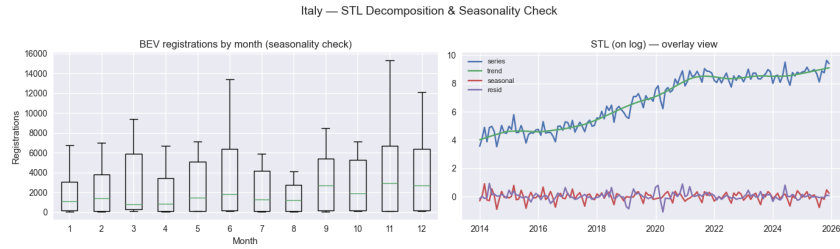


Figure 4.4: Seasonality analysis of BEV registrations in Italy

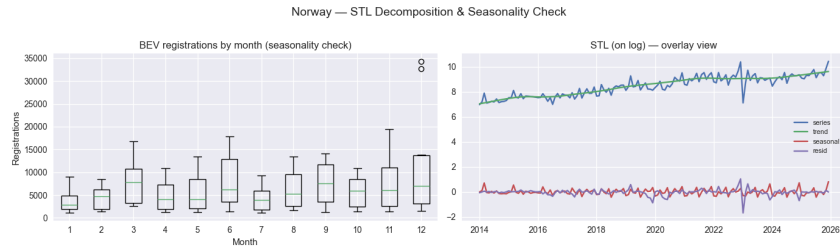


Figure 4.5: Seasonality analysis of BEV registrations in Norway

Both countries exhibit a strong upward trend in registrations. Italy shows a relatively stable seasonal pattern, while Norway displays more variability in its seasonal component.

The residual component from the decomposition reveals deviations from zero, indicating that not all variation is explained by trend and seasonality. This suggests that the residuals are not purely random and may contain additional structure that can be modeled.

To further investigate this, autocorrelation diagnostics (ACF and PACF) and stationarity tests (ADF and KPSS) are applied.

Table 4.1: Stationarity tests for BEV registration series

Country	Data Transformation	Test	Test Statistic	p-value	Lags	Observations
Italy	BEV levels	ADF	0.1041	0.9663	13	130
Italy	BEV levels	KPSS	1.5662	0.0100	8	—
Italy	log1p(BEV)	ADF	-0.5838	0.8746	12	131
Italy	log1p(BEV)	KPSS	1.6317	0.0100	8	—
Norway	BEV levels	ADF	0.5584	0.9865	5	138
Norway	BEV levels	KPSS	1.5729	0.0100	8	—
Norway	log1p(BEV)	ADF	-0.5894	0.8734	12	131
Norway	log1p(BEV)	KPSS	1.6456	0.0100	8	—

The results indicate that both series are non-stationary in levels, even after log transformation. Stationarity is only achieved after differencing. This behavior is consistent with the presence of a strong growth trend, which is typical in innovation diffusion processes.

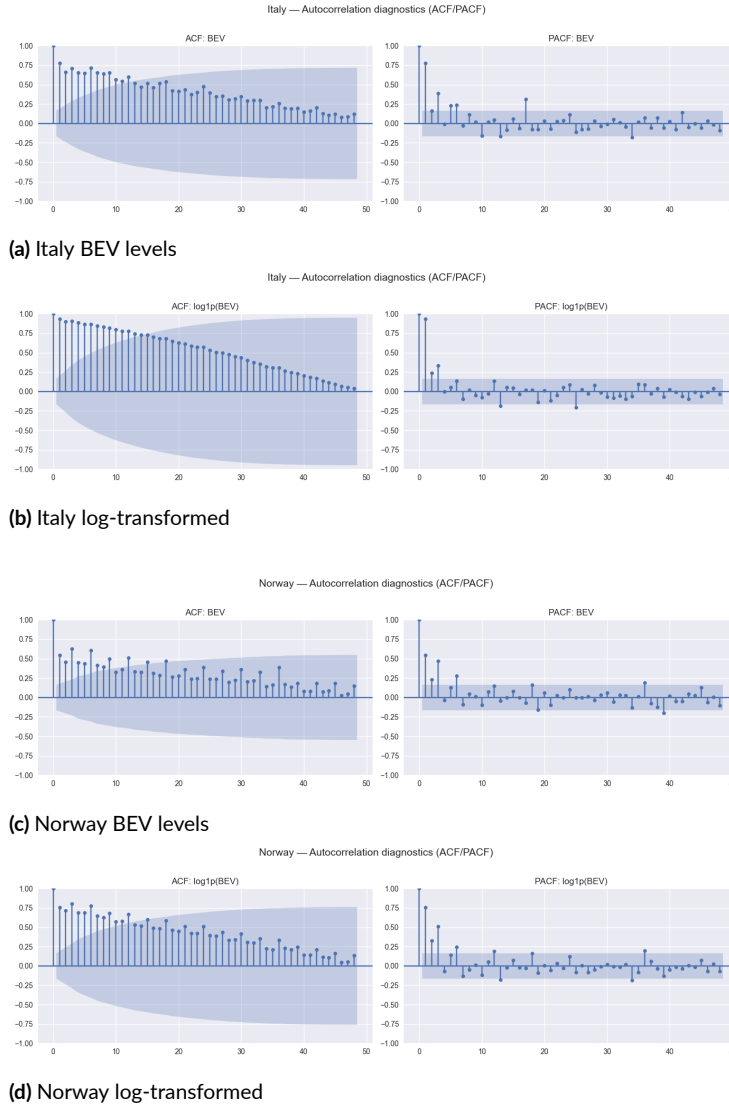


Figure 4.6: Autocorrelation diagnostics (ACF/PACF) for BEV registrations

The ACF and PACF plots show strong temporal dependence in both series. The ACF exhibits a slow decay across multiple lags, indicating persistence and possible non-stationarity. The PACF shows significant spikes at early lags, suggesting that past values have a strong influence on current observations.

These findings indicate that the series contain meaningful temporal structure beyond the main trend. As a result, the residual component cannot be treated as white noise and may benefit from additional modeling. This provides empirical motivation for incorporating residual time-series models alongside the diffusion framework.

4.2 ITALY RESULTS (LOG AND RAW): COMPARISON ACROSS RESIDUAL REFINEMENTS

This section evaluates whether residual modeling improves forecasting performance for the Italian market. In addition, different diffusion model specifications are compared to better understand BEV adoption dynamics.

Table 4.2 reports the forecasting results on the raw scale. The best performance is achieved by the Bass model combined with a Temporal Convolutional Network (TCN) residual model, which yields the lowest MAE and RMSE among all specifications.

Table 4.2: Forecast performance comparison on the raw scale

Scale	Model	MAE	RMSE
Raw	Bass	3610.39	4350.14
Raw	Bass + SARIMAX residual	2596.01	3408.79
Raw	Bass + NN residual LSTM	2019.38	2894.56
Raw	Bass + NN residual TCN	1783.60	2314.68
Raw	Bass + MH-TCN residual	3075.89	3877.89
Raw	Dynamic Market Potential	3298.25	4064.53
Raw	Dynamic Market Potential + SARIMAX residual	3365.75	4029.41
Raw	Dynamic Market Potential + NN residual LSTM	3071.66	3924.88
Raw	Dynamic Market Potential + NN residual TCN	3531.41	4017.17
Raw	Dynamic Market Potential + MH-TCN residual	9294.87	9605.98

Table 4.3 presents the results on the log-transformed scale. In this case, the best-performing model is the Bass specification combined with a multi-horizon TCN (MH-TCN) residual model, which achieves the lowest MAE and RMSE.

Table 4.3: Forecast performance comparison on the log scale

Scale	Model	MAE	RMSE
Log	Bass	0.476213	0.582629
Log	Bass + SARIMAX residual	0.444475	0.541680
Log	Bass + NN residual LSTM	0.873520	1.038778
Log	Bass + NN residual TCN	0.940554	1.033247
Log	Bass + MH-TCN residual	0.374711	0.444122
Log	Dynamic Market Potential	0.909505	1.051563
Log	Dynamic Market Potential + SARIMAX residual	0.923326	1.049884
Log	Dynamic Market Potential + NN residual LSTM	1.533037	1.717792
Log	Dynamic Market Potential + NN residual TCN	1.299086	1.401426
Log	Dynamic Market Potential + MH-TCN residual	1.556303	1.655546

The results clearly show that incorporating residual modeling improves forecasting accuracy compared to using diffusion models alone. On the raw scale, neural network-based residual models outperform the SARIMAX specification, with the TCN model providing the best results. On the log scale, the multi-horizon TCN achieves the highest accuracy, indicating that modeling short-term dynamics jointly across multiple horizons is particularly effective.

These findings suggest that nonlinear residual dynamics and seasonal effects play an important role in BEV adoption in Italy, and that neural models are better suited to capture these patterns than linear approaches.

In contrast, the Dynamic Market Potential specification does not improve forecasting performance in this case. In several instances, it even leads to worse results, particularly on the raw scale. This suggests that, for the Italian market, the added flexibility of a time-varying market potential does not translate into better predictive performance.

Overall, the results indicate that a standard Bass diffusion model combined with a flexible residual modeling approach, particularly using TCN-based architectures, provides the most effective forecasting framework for Italy.

4.3 NORWAY RESULTS (LOG AND RAW): COMPARISON ACROSS RESIDUAL REFINEMENTS

This section evaluates whether residual modeling improves forecasting performance for the Norwegian market. In addition, different diffusion model specifications are compared to analyze the behavior of BEV adoption.

Table 4.4 presents the results obtained using Norway’s raw data. The best-performing model on the raw scale is the Bass model combined with a multi-horizon Temporal Convolutional Network (MH-TCN) residual model, as it achieves the lowest MAE and RMSE.

Table 4.4: Forecast performance comparison on the raw scale (Experiment 2)

Model Family	Model	MAE	RMSE
Bass	Bass	2985.62	3728.83
	Bass + SARIMAX residual	3397.42	4257.15
	Bass + NN residual LSTM	2993.73	3787.05
	Bass + NN residual TCN	6355.96	8799.74
	Bass + MH-TCN residual	2687.95	3354.69
Dynamic Market Potential	Dynamic Market Potential	3511.06	4296.59
	+ SARIMAX residual	5160.46	7895.20
	+ NN residual LSTM	3854.86	4530.19
	+ NN residual TCN	3891.94	5510.13
	+ MH-TCN residual	2901.51	3571.00

Table 4.5 reports the results for the log-transformed data. In this case, the best-performing specification is the Dynamic Market Potential model combined with a multi-horizon Temporal Convolutional Network (MH-TCN) residual model, which achieves the lowest MAE and RMSE.

Table 4.5: Forecast performance comparison on the log scale (Experiment 2)

Model Family	Model	MAE	RMSE
Bass	Bass	0.4854	0.6109
	Bass + SARIMAX residual	0.4515	0.5766
	Bass + NN residual LSTM	3.2113	3.2729
	Bass + NN residual TCN	0.9573	1.1827
	Bass + MH-TCN residual	0.4431	0.5396
Dynamic Market Potential	Dynamic Market Potential	0.4108	0.6124
	+ SARIMAX residual	0.4051	0.6360
	+ NN residual LSTM	0.5716	0.8520
	+ NN residual TCN	0.3308	0.5248
	+ MH-TCN residual	0.3120	0.4790

For Norway, the results differ from those observed for Italy. Norway represents a more mature and stable market. In this setting, the Dynamic Market Potential model improves forecasting performance. When combined with neural residual models, particularly the multi-horizon TCN, it achieves the lowest forecasting errors.

These results suggest that dynamic diffusion models are more effective in markets where adoption evolves in a stable and predictable manner.

4.4 BEST MODEL SELECTION PER COUNTRY AND INTERPRETATION

Overall, the comparison between Italy and Norway highlights the importance of market maturity. In volatile markets, simpler diffusion models combined with neural residuals provide more robust forecasts. In mature markets, allowing the market potential to evolve over time leads to better results. Across both countries, neural residual modeling consistently enhances diffusion forecasts, supporting the central contribution of this thesis.

With this said, the models selection for Italy is Bass model together with a MH-TCN for the residual modelling while for Norway is Dynamic Market Potential with a MH-TCN for the residual modelling. The chosen results are in log-scale data and this justification will be exposed in Chapter 5.3.

4.5 COMPUTATIONAL COST VS ACCURACY TRADE-OFF

Although neural models achieve the best forecasting accuracy, it is important to assess whether their use is justified given the additional computational cost. For this reason, training time is compared with improvements in MAE for both countries. Table 4.6 presents results for Bass-based models, while Table 4.7 reports results for Dynamic Market Potential models.

Table 4.6: Forecast improvement and computational cost for Bass-based models

Model	MAE	MAE Diff	Train time (s)
Bass	0.476213	—	< 0.001
Bass + SARIMAX	0.444475	7.14%	0.483
Bass + MH-TCN	0.374711	15.70%	42.342

Table 4.7: Forecast improvement and computational cost for Dynamic Market Potential models

Model	MAE	MAE Diff	Train time (s)
Dynamic Market Potential	0.410848	—	0.552
Dynamic Market Potential + SARIMAX	0.405099	1.42%	0.677
Dynamic Market Potential + MH-TCN	0.312021	22.98%	73.503

When comparing models, log-scale errors are used to evaluate forecasting accuracy in relative terms. This allows for a fair comparison across different stages of the diffusion process.

While the multi-horizon TCN residual model achieves the lowest forecast errors, the improvement over the SARIMAX model is moderate when compared to the substantial increase in training time. For smaller datasets, simpler residual models may provide a better cost–benefit trade-off. In contrast, for longer forecasting horizons, smoother markets, or larger datasets, neural residual models become more advantageous due to their higher predictive accuracy.

Computational trade-off:

SARIMAX: strong gains, negligible cost

MH-TCN: best accuracy, higher training cost. Worth it mainly for long horizons or larger datasets

4.6 FORECAST COMPARISON

Finally, it is important to visually assess the forecasting results. This helps determine whether the model outputs are consistent with the observed data and the context of the analyzed markets.

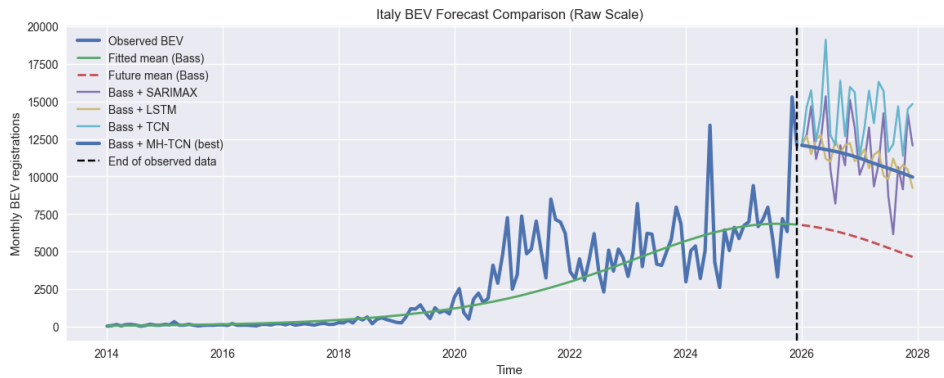


Figure 4.7: Forecast produced by the Bass model with residual refinement for Italy. The figure compares observed BEV registrations with the predicted trajectory over the forecasting horizon.

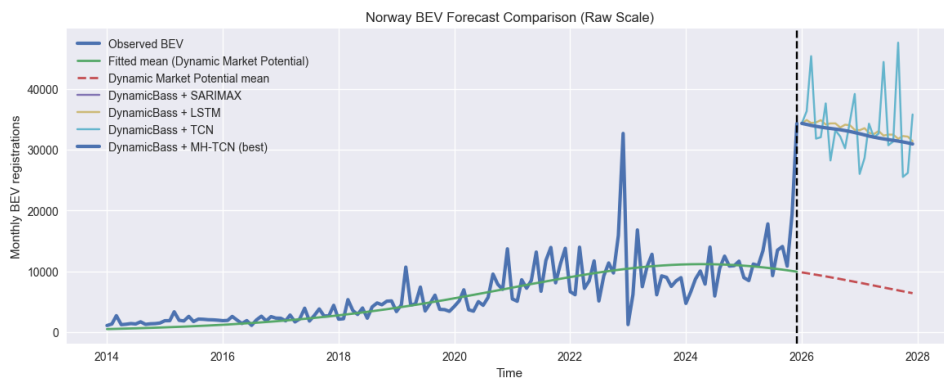


Figure 4.8: Forecast produced by the Dynamic Market Potential model with residual refinement for Norway. The predicted trajectory is compared with observed BEV registrations over the forecasting horizon.

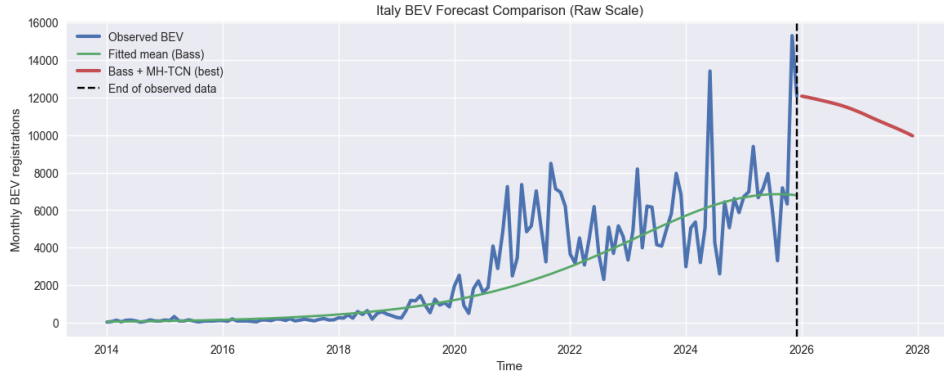


Figure 4.9: Best Forecast model produced by the Bass model with MH-TCN for Italy.

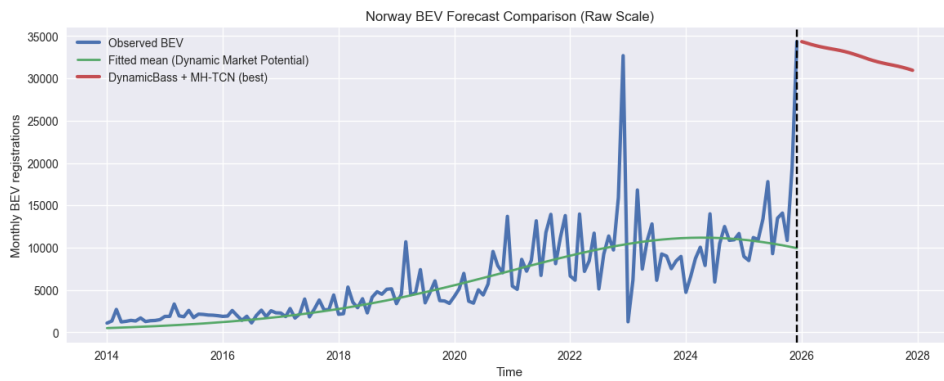


Figure 4.10: Best Forecast model produced by the Dynamic Market Potential model with MH-TCN for Norway.

The results shown in Figure 4.7 and Figure 4.8 present the final forecast comparisons for Italy and Norway on the raw scale. Each figure includes the observed data, the fitted diffusion mean, the projected diffusion trend, and the forecasts obtained by combining diffusion models with different residual correction methods.

For Italy, the Bass diffusion model captures the overall long-term adoption trend but does not fully represent the strong short-term fluctuations observed in the data. The inclusion of residual models improves the ability to capture these variations. Among the approaches tested, neural residual models, particularly the multi-horizon TCN, produce forecasts that better reflect recent volatility. These visual results are consistent with the quantitative findings, where neural models achieve the lowest forecast errors.

For Norway, the Dynamic Market Potential model provides a strong baseline fit, reflecting a more mature and stable adoption process. In this case, the diffusion model explains most

of the long-term structure, and the differences between residual modeling approaches are less pronounced than in Italy. Nevertheless, residual models still improve the representation of short-term deviations, with neural approaches offering additional flexibility.

Overall, the forecast comparison highlights the complementary roles of the two model components. Diffusion models capture long-term adoption dynamics, while residual models account for short-term variations that are not explained by the structural trend.

5

Discussion

This chapter interprets the results achieved in Chapter 4, analysis of the main observations found, what do the results mean and what could be the implementation limitations of these types of models.

5.1 RESIDUAL LEARNING IMPROVEMENTS

From the results for both markets, innovation diffusion models such as the Bass model and the Dynamic Market Potential model capture the long-term adoption trend well [4] [6], but leave structured residual patterns that reflect short-term fluctuations not explained by the diffusion component alone. As shown in Chapter 4, the residuals exhibit autocorrelation and seasonal patterns, and cannot be treated as white noise.

The empirical results confirm that explicitly modeling these residuals delivers meaningful accuracy gains. For Italy, incorporating MH-TCN residuals into the Bass baseline reduced MAE by 15.7% on the log scale, while SARIMAX refinement achieved only 7.1%. This gap suggests that the short-term dynamics left unexplained by the Bass model are genuinely nonlinear, and that a linear seasonal filter recovers only part of the available structure. For Norway, the Dynamic Market Potential model combined with MH-TCN residuals achieved a log-scale MAE of 0.312, compared to 0.411 for the unrefined baseline, a reduction of roughly 24%. In both markets, SARIMAX alone produced smaller gains, reinforcing the same conclusion: once the

structural trend is well specified, the remaining residual dynamics are better described by a flexible nonlinear model than by a linear autoregressive one.

Among the neural approaches, the TCN model performs consistently well in both markets. This is because it learns from a sliding window of past residuals and is able to pick up recurring seasonal patterns effectively. The multi-horizon TCN goes one step further: instead of predicting one month at a time and feeding that prediction back into the next step, it predicts all 24 future months at once. This avoids the build-up of errors over the forecasting horizon, which is likely why it achieves the best results across both countries [11].

Overall, these results show that adding a residual model on top of a diffusion baseline consistently improves forecast accuracy. The improvement is larger when the market is volatile and the residuals are harder to explain with a simple linear model, as was the case for Italy.

5.2 WHY DYNAMIC MARKET POTENTIAL HELPED NORWAY MORE THAN ITALY

Although both markets show promising results when combining structural diffusion models with neural residual models, the results differ regarding whether a standard Bass model or a model with dynamic market potential provides a better fit. In the case of Norway, the Dynamic Market Potential model improves forecasting performance, whereas for Italy the standard Bass model performs better when combined with neural residual models.

To better understand these differences, an analysis of market maturity is necessary. Norway's BEV market began developing much earlier than Italy's. Today, the majority of vehicles circulating in the country are electric, making BEVs the dominant and most established technology in the automotive market. As a result, adoption levels are high and supported by stable long-term policies. In addition to government incentives, social influence plays an important role: consumers are highly aware of electric mobility, and BEVs have become normalized within the country's culture and mobility habits. Furthermore, Norway has developed a mature charging infrastructure that supports large-scale adoption. These conditions suggest that in recent years the diffusion process has become smoother, allowing the dynamic market size to be more clearly identified.

In contrast, Italy is still far from reaching a similar level of adoption. Social influence, consumer mentality, and cultural acceptance of electric vehicles are still evolving. In addition, charging infrastructure and policy support remain less developed and less stable compared

to Norway. The data also show higher volatility, lower adoption levels, and frequent policy changes affecting the market. In such volatile environments, it becomes more difficult to estimate reliable parameters for dynamic diffusion models.

In conclusion, the results suggest that dynamic diffusion models perform better in stable and mature markets. This helps explain why forecasting results improved for Norway when using a dynamic market potential specification, while for Italy the best-performing models relied on the standard Bass framework with a constant market potential.

5.3 LOG VS RAW: WHAT IS FAIR TO COMPARE AND HOW TO DECIDE

The rapid and continuous growth in BEV adoption leads to wide variances and large-scale differences in BEV adoption from the early period through to the late period. By evaluating the models on the logarithm of the scale, the variances stabilize and the errors are assessed on a relative basis, thus allowing for the performance of the forecasts to be assessed on an equal basis throughout the entire diffusion process. This results in fair comparisons between the models and a more reliable determination of residual learning, with the models ultimately being interpreted on the raw scale [8].

The models were selected using the log scale error metrics as they measure the relative error of the forecast rather than the absolute error. Because of the rapid growth in the BEV adoption raw error in the early stages of adoption is virtually insignificant in comparison to raw error at later stages of adoption and therefore can impact the evaluation, de-emphasizing the early stages of BEV adoption.

As a result, the log scale allows for early and late stages of BEV adoption to be treated equally in terms of their relative importance, providing a more fair comparison between models, while still providing raw error metrics for understanding the real-world implications of BEV adoption through the entire adoption process and allowing for the best overall model from the full data set.

5.4 PRACTICAL INTERPRETATION FOR FORECASTING AND POLICY/BUSINESS USE

This section discusses the relevance of the results in a real-world context. Forecasting technology adoption is important not only for academic purposes but also for policymakers, industry stakeholders, and practitioners involved in market planning and forecasting. For policymakers, these forecasting results can support infrastructure planning, subsidy design, and charging network investments.

Understanding how quickly BEV adoption is likely to grow helps governments plan the expansion of charging infrastructure and evaluate the long-term impact of incentives or regulatory measures aimed at accelerating the transition toward electric mobility.

For industry stakeholders, accurate forecasts are important for production planning, EV charging deployment, and battery demand forecasting. Automotive manufacturers and energy providers rely on demand forecasts to make strategic decisions about production capacity, supply chains, and infrastructure investments.

Finally, the results also provide useful insights for forecasting practice. Instead of choosing between structural models and machine learning approaches, this study shows that combining both can lead to improved forecasting performance. Structural diffusion models capture the long-term adoption dynamics, while machine learning models can learn the remaining residual patterns. This hybrid approach highlights that residuals may contain meaningful information that can be modeled and exploited to improve predictions.

5.5 LIMITATIONS

There are several limitations to this study that should be considered when interpreting the results. Addressing these limitations in future work could lead to a more robust and comprehensive investigation. One limitation is the size of the dataset used in this study. The available data consists of monthly observations with a relatively limited historical range. For diffusion models, it would be beneficial to observe the entire adoption cycle from the early introduction of the technology to later stages of market maturity. In addition, higher-frequency data (for example weekly data) could potentially provide more detailed information about short-term dynamics. Expanding the analysis to include additional countries could also strengthen the robustness of the conclusions.

Another limitation relates to parameter identifiability in diffusion models. Dynamic Bass models in particular may suffer from instability in parameter estimation, as different parameter combinations can sometimes produce similar model fits. This can create challenges when interpreting parameter values and evaluating model performance.

A further limitation is the presence of policy shocks and external interventions. Real-world EV markets are heavily influenced by policy measures such as subsidies, regulatory changes, and government incentives. These events can introduce structural breaks in the data that are difficult to capture using standard diffusion models. Incorporating policy variables explicitly into the models could potentially improve forecasting performance.

Finally, diffusion models often assume relatively stable long-term adoption dynamics, while real markets may evolve in unexpected ways. Changes in technology, regulation, or consumer behavior can alter adoption trajectories and affect long-term forecasts.

Despite these limitations, the results of this study provide useful evidence on how structural diffusion models and residual learning approaches can be combined to improve forecasting performance in innovation diffusion contexts.

6

Conclusion

6.1 SUMMARY OF FINDINGS

In this thesis, we investigated how residual modeling techniques can improve existing innovation diffusion models. Specifically, the objective was to determine whether combining structural diffusion models with machine learning methods leads to improved forecasting accuracy in the context of technology adoption. To address this, monthly registrations of Battery Electric Vehicles (BEVs) in Italy and Norway were used to evaluate multiple forecasting approaches.

The baseline models considered were diffusion models, namely the Bass model and a modified version incorporating dynamic market potential. These models are well suited to capturing long-term adoption patterns of new technologies. However, they often fail to account for short-term variability observed in real-world data. To address this limitation, several residual refinement techniques were implemented, including SARIMAX models and neural approaches such as LSTM and TCN.

The results show that incorporating residual modeling significantly improves forecasting performance compared to using diffusion models alone. Among the approaches tested, the multi-horizon TCN residual model achieved the lowest forecast errors.

In addition, the results highlight important differences between markets. In Italy, higher volatility and stronger policy-driven dynamics make residual modeling particularly important for improving forecast accuracy. In contrast, Norway represents a more stable and mature mar-

ket, where the Dynamic Market Potential model provides a stronger structural baseline.

Overall, the findings provide strong evidence that hybrid models combining structural diffusion models with residual learning techniques offer an effective approach for forecasting technology adoption.

6.2 ANSWER TO RESEARCH QUESTIONS

The main research questions posed in Chapter 1 can now be answered directly. First, diffusion model residuals do contain systematic temporal structure, as confirmed by the autocorrelation diagnostics and STL decomposition in Chapter 4. Second, SARIMAX refinement does improve diffusion forecasts, though the gains are moderate. Third, neural residual models provide additional improvements beyond linear refinement, particularly in volatile markets such as Italy. Fourth, the performance gain of the MH-TCN is meaningful but comes at a significant computational cost, making it most justified for longer horizons or larger datasets. Fifth, market characteristics do matter: dynamic diffusion models perform better in stable markets like Norway, while simpler baselines are more robust in volatile ones like Italy.

6.3 RECOMMENDATIONS AND FUTURE WORK

Based on the results of this study, several directions for future research can be suggested.

First, future work could explore larger datasets and a wider range of markets. Including additional countries or longer historical periods would provide more robust evidence on the effectiveness of hybrid diffusion–machine learning approaches.

Second, incorporating additional explanatory variables could improve model performance. Variables related to policy incentives, charging infrastructure development, or fuel prices may help explain some of the short-term fluctuations observed in BEV adoption.

Third, future research could investigate alternative neural architectures for residual modeling. For example, transformer-based models or probabilistic forecasting approaches may provide further improvements in predictive performance.

Finally, future studies could explore scenario-based forecasting approaches, where diffusion models are combined with policy scenarios or technological developments to simulate different adoption pathways.

This research contributes to the literature on innovation diffusion forecasting by demonstrating that residual learning can effectively complement structural diffusion models. The

proposed work offers a promising direction for improving the forecasting of emerging technologies such as electric vehicles.

A

Code

A.1 BASS DIFFUSION MODEL

```
def bass_flow(p, q, m, T):  
    """Bass expected flow (monthly new adopters) for t=1..T"""  
    s = np.zeros(T, dtype=float)  
    Y_cum = 0.0  
    for t in range(T):  
        F_prev = Y_cum / m  
        s_t = m * (p + q * F_prev) * (1 - F_prev)  
        s_t = max(s_t, 1e-12)  
        s[t] = s_t  
        Y_cum = min(Y_cum + s_t, m * (1 - 1e-9))  
    return s  
  
def fit_bass_on_loglp_flow(ts_bev: pd.Series, train_end: str):  
    """Fit Bass by minimizing MSE between loglp(observed flow) and  
    loglp(predicted flow)"""  
    ts_bev = ts_bev.dropna()  
    train_end = pd.to_datetime(train_end)  
    train = ts_bev[ts_bev.index <= train_end]
```

```

y = np.log1p(train.values)
bev = train.values
T = len(train)

cum_obs = float(np.sum(bev))
p0, q0 = 0.01, 0.4
m0 = max(cum_obs * 1.5, 1.0)

bounds = [
    (1e-6, 1.0),          # p
    (1e-6, 5.0),          # q
    (cum_obs * 1.01, cum_obs * 200.0 + 1e6) # m
]

def obj(theta):
    p, q, m = theta
    s_hat = bass_flow(p, q, m, T)
    y_hat = np.log1p(s_hat)
    return float(np.mean((y - y_hat) ** 2))

res = minimize(obj, x0=np.array([p0, q0, m0]), bounds=bounds, method="L-BFGS-B")
p_hat, q_hat, m_hat = res.x
return {"p": float(p_hat), "q": float(q_hat), "m": float(m_hat),
        "success": bool(res.success), "fun": float(res.fun)}

def bass_predict_log1p_train(ts_bev: pd.Series, params: dict, train_end: str):
    train_end = pd.to_datetime(train_end)
    train = ts_bev[ts_bev.index <= train_end].dropna()
    T = len(train)
    s_hat = bass_flow(params["p"], params["q"], params["m"], T)
    return pd.Series(np.log1p(s_hat), index=train.index)

def bass_forecast_log1p(ts_bev: pd.Series, params: dict, train_end: str,
                        horizon: int):

```

```

train_end = pd.to_datetime(train_end)
train = ts_bev[ts_bev.index <= train_end].dropna()
T_train = len(train)

s_train_hat = bass_flow(params["p"], params["q"], params["m"], T_train)
Y_cum = float(np.sum(s_train_hat))
Y_cum = min(Y_cum, params["m"] * (1 - 1e-9))

s_fore = np.zeros(horizon, dtype=float)
for h in range(horizon):
    F_prev = Y_cum / params["m"]
    s_h = params["m"] * (params["p"] + params["q"] * F_prev) * (1 - F_prev)
    s_h = max(s_h, 1e-12)
    s_fore[h] = s_h
    Y_cum = min(Y_cum + s_h, params["m"] * (1 - 1e-9))

start = (train.index.max() +
pd.DateOffset(months=1)).to_period("M").to_timestamp()
idx = pd.date_range(start=start, periods=horizon, freq="MS")
return pd.Series(np.log1p(s_fore), index=idx)

## RAW

def fit_bass_on_raw_flow_wmse(ts_bev: pd.Series, train_end: str, eps=50.0):
    """
    Weighted MSE: weights = 1/(y+eps)
    """
    ts_bev = ts_bev.dropna()
    train_end = pd.to_datetime(train_end)
    train = ts_bev[ts_bev.index <= train_end]

    y = train.values.astype(float)
    T = len(train)

```

```

cum_obs = float(np.sum(y))
p0, q0 = 0.01, 0.4
m0 = max(cum_obs * 1.5, 1.0)

bounds = [
    (1e-6, 1.0),
    (1e-6, 5.0),
    (cum_obs * 1.01, cum_obs * 200.0 + 1e6)
]

w = 1.0 / (y + eps)

def obj(theta):
    p, q, m = theta
    s_hat = bass_flow(p, q, m, T)
    return float(np.mean(w * (y - s_hat) ** 2))

res = minimize(obj, x0=np.array([p0, q0, m0]), bounds=bounds, method="L-BFGS-B")
p_hat, q_hat, m_hat = res.x
return {
    "p": float(p_hat), "q": float(q_hat), "m": float(m_hat),
    "success": bool(res.success), "fun": float(res.fun)
}

def bass_predict_train_raw(ts_bev: pd.Series, params: dict, train_end: str):
    train_end = pd.to_datetime(train_end)
    train = ts_bev[ts_bev.index <= train_end].dropna()
    T = len(train)
    s_hat = bass_flow(params["p"], params["q"], params["m"], T)
    return pd.Series(s_hat, index=train.index)

def bass_forecast_raw(ts_bev: pd.Series, params: dict, train_end: str,
horizon: int):
    train_end = pd.to_datetime(train_end)

```

```

train = ts_bev[ts_bev.index <= train_end].dropna()
T_train = len(train)

s_train_hat = bass_flow(params["p"], params["q"], params["m"], T_train)
Y_cum = float(np.sum(s_train_hat))
Y_cum = min(Y_cum, params["m"] * (1 - 1e-9))

s_fore = np.zeros(horizon, dtype=float)
for h in range(horizon):
    F_prev = Y_cum / params["m"]
    s_h = params["m"] * (params["p"] + params["q"] * F_prev) * (1 - F_prev)
    s_h = max(s_h, 1e-12)
    s_fore[h] = s_h
    Y_cum = min(Y_cum + s_h, params["m"] * (1 - 1e-9))

start = (train.index.max() +
pd.DateOffset(months=1)).to_period("M").to_timestamp()
idx = pd.date_range(start=start, periods=horizon, freq="MS")
return pd.Series(s_fore, index=idx)

```

A.2 DYNAMIC MARKET POTENTIAL

```

def M_exp(t, M0, gamma):
    """Exponential market potential:  $M_t = M_0 * \exp(\gamma * t)$ """
    return M0 * np.exp(gamma * t)

def M_logistic(t, M_min, M_max, k, t0):
    return M_min + (M_max - M_min) / (1.0 + np.exp(-k * (t - t0)))

def bass_dynamic_flow(p, q, M_t):
    T = len(M_t)
    s = np.zeros(T, dtype=float)
    Y = 0.0

```

```

for t in range(T):
    Mt = max(M_t[t], 1e-9)
    Mt_prev = max(M_t[t-1], 1e-9) if t > 0 else Mt

    F_prev = min(max(Y / Mt_prev, 0.0), 1.0)
    s_t = (p + q * F_prev) * (Mt - Y)

    s_t = max(s_t, 1e-12)
    s[t] = s_t
    Y = min(Y + s_t, Mt * (1 - 1e-9))

return s

def fit_dynamic_bass_logisticM(ts_flow: pd.Series, train_end: str,
                               k_max=0.05, t0_lo=0.1, t0_hi=0.9,
                               reg=1e-8):
    ts_flow = ts_flow.dropna().astype(float)
    train_end = pd.to_datetime(train_end)
    train = ts_flow.loc[:train_end]
    y = train.values
    T = len(train)
    t = np.arange(T, dtype=float)

    cum_obs = float(y.sum())
    y_max = float(y.max())

    #init
    p0, q0 = 0.001, 0.08

    M_min0 = max(cum_obs * 0.9, y_max * 12)
    M_max0 = max(cum_obs * 1.2, y_max * 36)

    k0 = min(0.02, k_max * 0.5)
    t00 = T * 0.5

```



```

x0 = np.array([p0, q0, M_min0, M_max0, k0, t00], dtype=float)

# bounds
t0_lower = t0_lo * T
t0_upper = t0_hi * T

bounds = [
    (1e-5, 0.2),          # p
    (1e-5, 1.5),          # q
    (cum_obs * 0.5, cum_obs*2), # M_min
    (cum_obs * 1.05, cum_obs*8), # M_max
    (1e-4, k_max),        # k smaller upper bound
    (t0_lower, t0_upper)  # t0
]

def obj(theta):
    p, q, M_min, M_max, k, t0 = theta
    if M_max <= M_min:
        return 1e30

    M_t = M_logistic(t, M_min, M_max, k, t0)
    s_hat = bass_dynamic_flow(p, q, M_t)

    mse = np.mean((y - s_hat) ** 2)

    pen = reg * ((k / k_max) ** 2 + ((t0 - T/2) / (T/2)) ** 2)

    return float(mse + pen)

res = minimize(obj, x0=x0, bounds=bounds, method="L-BFGS-B")

p, q, M_min, M_max, k, t0 = res.x
M_t = M_logistic(t, M_min, M_max, k, t0)

```

```

s_hat = bass_dynamic_flow(p, q, M_t)

yhat_train = pd.Series(s_hat, index=train.index)
resid_train = train - yhat_train

return {
    "p": float(p), "q": float(q),
    "M_min": float(M_min), "M_max": float(M_max),
    "k": float(k), "t0": float(t0),
    "success": bool(res.success), "fun": float(res.fun),
    "yhat_train": yhat_train,
    "resid_train": resid_train,
}

def forecast_dynamic_bass_logisticM(ts_flow: pd.Series, fit: dict,
train_end: str, horizon: int):
    ts_flow = ts_flow.dropna().astype(float)
    train_end = pd.to_datetime(train_end)
    train = ts_flow.loc[:train_end]
    T_train = len(train)

    p, q = fit["p"], fit["q"]
    M_min, M_max, k, t0 = fit["M_min"], fit["M_max"], fit["k"], fit["t0"]

    T_total = T_train + horizon
    t_all = np.arange(T_total, dtype=float)
    M_all = M_logistic(t_all, M_min, M_max, k, t0)

    s_all = bass_dynamic_flow(p, q, M_all)
    s_fore = s_all[T_train:]

    start = (train.index.max() +
pd.DateOffset(months=1)).to_period("M").to_timestamp()
    idx = pd.date_range(start=start, periods=horizon, freq="MS")

```

```
return pd.Series(s_fore, index=idx)
```

A.3 SARIMAX

```
from statsmodels.tsa.statespace.sarimax import SARIMAX

def fit_residual_sarimax(resid_train: pd.Series):
    # arma 1,0,1 sarixma 1,0,0
    model = SARIMAX(
        resid_train,
        order=(1,0,1),
        seasonal_order=(1,0,0,12),
        trend="n",
        enforce_stationarity=False,
        enforce_invertibility=False
    )
    res = model.fit(dispatch=False)
    return res

sarimax_fit = fit_residual_sarimax(resid_train)

resid_fore = sarimax_fit.get_forecast(steps=H).predicted_mean
resid_fore.index = yhat_fore.index

# Final forecast = diffusion mean + residual forecast
y_fore_sarimax = yhat_fore + resid_fore

print("Done: diffusion + SARIMAX residual baseline")

#RAW
```

```

sarimax_fit_raw = fit_residual_sarimax(resid_train_raw)

resid_fore_raw = sarimax_fit_raw.get_forecast(steps=H).predicted_mean
resid_fore_raw.index = yhat_fore_raw.index
# Final forecast = diffusion mean + residual forecast
y_fore_sarimax_raw = yhat_fore_raw + resid_fore_raw
print("Done: diffusion + SARIMAX residual baseline - raw")

## Norway

sarimax_fit_2 = fit_residual_sarimax(resid_train_2)
resid_fore_2 = sarimax_fit_2.get_forecast(steps=H).predicted_mean
resid_fore_2.index = yhat_fore_2.index
y_fore_sarimax_2 = yhat_fore_2 + resid_fore_2
print("Done: diffusion + SARIMAX residual baseline - Norway")

#RAW Norway

sarimax_fit_raw_2 = fit_residual_sarimax(resid_train_raw_2)
resid_fore_raw_2 = sarimax_fit_raw_2.get_forecast(steps=H).predicted_mean
resid_fore_raw_2.index = yhat_fore_raw_2.index
y_fore_sarimax_raw_2 = yhat_fore_raw_2 + resid_fore_raw_2
print("Done: diffusion + SARIMAX residual baseline - raw - Norway")

```

A.4 DIFFUSION + NEURAL RESIDUAL MODELS

```

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

class ResidualDataset(Dataset):

```

```

def __init__(self, resid: pd.Series, lookback=24):
    self.lookback = lookback
    self.resid = resid.values.astype(np.float32)
    self.months = resid.index.month.astype(np.int64)
    self.X, self.m, self.y = [], [], []
    for i in range(lookback, len(self.resid)):
        self.X.append(self.resid[i-lookback:i])
        self.m.append(self.months[i-1])
        self.y.append(self.resid[i])
    self.X = np.stack(self.X)
    self.m = np.array(self.m)
    self.y = np.array(self.y).astype(np.float32)

def __len__(self): return len(self.y)

def __getitem__(self, idx):
    return (
        torch.from_numpy(self.X[idx]).unsqueeze(-1),
        torch.tensor(self.m[idx]-1),
        torch.tensor(self.y[idx])
    )

class ResidualLSTM(nn.Module):
    def __init__(self, hidden=64, month_emb=8):
        super().__init__()
        self.month_emb = nn.Embedding(12, month_emb)
        self.lstm = nn.LSTM(input_size=1, hidden_size=hidden, batch_first=True)
        self.head = nn.Sequential(
            nn.Linear(hidden + month_emb, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
        )

    def forward(self, x_seq, month_idx):

```

```

_, (h, _) = self.lstm(x_seq)
h = h[-1]
m = self.month_emb(month_idx)
out = self.head(torch.cat([h, m], dim=1))
return out.squeeze(-1)

```

TCN

```

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

```

```

class ResidualTCNDataset(Dataset):

```

```

    """

```

```

    Predict e_t from past residuals + calendar features.

```

```

    what comes from inputs:

```

```

    - lookback

```

```

    - lag12 residual

```

```

    - month sin/cos

```

```

    """

```

```

def __init__(self, resid: pd.Series, lookback=36):

```

```

    resid = resid.dropna()

```

```

    self.lookback = lookback

```

```

    r = resid.values.astype(np.float32)

```

```

    months = resid.index.month.astype(np.int64)

```

```

    X = []

```

```

    y = []

```

```

    for i in range(lookback, len(r)):

```

```

        window = r[i-lookback:i]

```

```

        m = months[i]

```

```

        m_sin = np.sin(2*np.pi*m/12.0)

```

```

        m_cos = np.cos(2*np.pi*m/12.0)

```

```

        lag12 = r[i-12] if i-12 >= 0 else 0.0

```

```

        X.append((window, lag12, m_sin, m_cos))
        y.append(r[i])

    self.y = torch.tensor(np.array(y, dtype=np.float32))
    self.window =
    torch.tensor(np.stack([x[0] for x in X]).astype(np.float32))
    self.lag12 =
    torch.tensor(np.array([x[1] for x in X], dtype=np.float32)).unsqueeze(-1)
    self.m_sin =
    torch.tensor(np.array([x[2] for x in X], dtype=np.float32)).unsqueeze(-1)
    self.m_cos =
    torch.tensor(np.array([x[3] for x in X], dtype=np.float32)).unsqueeze(-1)

def __len__(self):
    return len(self.y)

def __getitem__(self, idx):
    w = self.window[idx].unsqueeze(-1)
    feats = torch.cat([self.lag12[idx],
        self.m_sin[idx], self.m_cos[idx]], dim=-1)
    return w, feats, self.y[idx]

class CausalConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch, kernel=3, dilation=1, dropout=0.1):
        super().__init__()
        self.pad = (kernel - 1) * dilation
        self.conv = nn.Conv1d(in_ch, out_ch, kernel_size=kernel, dilation=dilation)
        self.act = nn.ReLU()
        self.drop = nn.Dropout(dropout)
        self.res = (nn.Conv1d(in_ch, out_ch, 1) if in_ch != out_ch
            else nn.Identity())

    def forward(self, x):
        x_padded = nn.functional.pad(x, (self.pad, 0))

```

```

        h = self.conv(x_padded)
        h = self.act(h)
        h = self.drop(h)
        return h + self.res(x)

class ResidualTCN(nn.Module):
    def __init__(self, hidden=32, levels=4, kernel=3, dropout=0.1):
        super().__init__()
        layers = []
        in_ch = 1
        for i in range(levels):
            layers.append(CausalConvBlock(in_ch, hidden,
                                           kernel=kernel, dilation=2**i, dropout=dropout))
            in_ch = hidden
        self.tcn = nn.Sequential(*layers)
        self.head = nn.Sequential(
            nn.Linear(hidden + 3, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
        )

    def forward(self, window, extra_feats):
        #window: (B, L, 1)
        x = window.permute(0, 2, 1)
        h = self.tcn(x)
        h_last = h[:, :, -1]
        out = self.head(torch.cat([h_last, extra_feats], dim=1))
        return out.squeeze(-1)

def train_tcn(resid_train: pd.Series, lookback=36, epochs=500,
              batch_size=16, lr=1e-3):
    ds = ResidualTCNDataset(resid_train, lookback=lookback)
    dl = DataLoader(ds, batch_size=batch_size, shuffle=True, drop_last=True)

```



```
model = ResidualTCN(hidden=32, levels=4, kernel=3, dropout=0.15).to(DEVICE)
```

```
opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-4)
```

```
loss_fn = nn.MSELoss()
```

```
for ep in range(epochs):
```

```
    model.train()
```

```
    total = 0.0
```

```
    for w, feats, y in dl:
```

```
        w = w.to(DEVICE)
```

```
        feats = feats.to(DEVICE)
```

```
        y = y.to(DEVICE)
```

```
        pred = model(w, feats)
```

```
        loss = loss_fn(pred, y)
```

```
        opt.zero_grad()
```

```
        loss.backward()
```

```
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
```

```
        opt.step()
```

```
        total += float(loss.item())
```

```
    if (ep+1) % 100 == 0:
```

```
        print(f"Epoch {ep+1}/{epochs} loss={total/len(dl):.6f}")
```

```
return model
```

```
@torch.no_grad()
```

```
def forecast_tcn(model, resid_train: pd.Series, start_date,  
horizon=12, lookback=36):
```

```
    model.eval()
```

```
    history = resid_train.values.astype(np.float32).tolist()
```

```
    idx =
```

```

pd.date_range(start=pd.to_datetime(start_date), periods=horizon, freq="MS")

preds = []
for d in idx:
    m = d.month
    m_sin = np.sin(2*np.pi*m/12.0)
    m_cos = np.cos(2*np.pi*m/12.0)
    lag12 = history[-12] if len(history) >= 12 else 0.0

    window = np.array(history[-lookback:],
dtype=np.float32).reshape(1, lookback, 1)
    feats = np.array([[lag12, m_sin, m_cos]], dtype=np.float32)

    w_t = torch.from_numpy(window).to(DEVICE)
    f_t = torch.from_numpy(feats).to(DEVICE)

    r_hat = model(w_t, f_t).cpu().numpy()[0]
    preds.append(float(r_hat))
    history.append(float(r_hat))

return pd.Series(preds, index=idx)

```

A.5 MH-TCN

```

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def month_sin_cos(month):
    msin = np.sin(2*np.pi*month/12.0)
    mcos = np.cos(2*np.pi*month/12.0)

```

```
return msin, mcos
```

```
class ResidualMultiHorizonDataset(Dataset):
```

```
    """
```

```
    Input: residual window of length L (past)
```

```
    Output: next H residuals (direct multi-step)
```

```
    Features: month sin/cos for each horizon step (H x 2)
```

```
    """
```

```
def __init__(self, resid: pd.Series, lookback=36, horizon=24):
```

```
    resid = resid.dropna()
```

```
    self.lookback = lookback
```

```
    self.horizon = horizon
```

```
    r = resid.values.astype(np.float32)
```

```
    idx = resid.index
```

```
    X = []
```

```
    F = []
```

```
    Y = []
```

```
    for t in range(lookback, len(r) - horizon):
```

```
        window = r[t-lookback:t]
```

```
        future = r[t:t+horizon]
```

```
        future_dates = idx[t:t+horizon]
```

```
        feats = []
```

```
        for d in future_dates:
```

```
            ms, mc = month_sin_cos(d.month)
```

```
            feats.append([ms, mc])
```

```
        feats = np.array(feats, dtype=np.float32)
```

```
    X.append(window)
```

```
    F.append(feats)
```

```
    Y.append(future)
```

```

self.X = torch.tensor(np.stack(X), dtype=torch.float32)
self.F = torch.tensor(np.stack(F), dtype=torch.float32)
self.Y = torch.tensor(np.stack(Y), dtype=torch.float32)

def __len__(self): return self.Y.shape[0]
def __getitem__(self, i):
    return self.X[i].unsqueeze(-1), self.F[i], self.Y[i]

class MultiHorizonTCN(nn.Module):
    """
    TCN encodes past residual window giving us a context vector
    """
    def __init__(self, hidden=32, levels=4, kernel=3, dropout=0.15, horizon=24):
        super().__init__()
        self.horizon = horizon

    def causal_block(in_ch, out_ch, dilation):
        pad = (kernel - 1) * dilation
        return nn.Sequential(
            nn.ConstantPad1d((pad, 0), 0.0),
            nn.Conv1d(in_ch, out_ch, kernel_size=kernel, dilation=dilation),
            nn.ReLU(),
            nn.Dropout(dropout),
        )

    layers = []
    in_ch = 1
    for i in range(levels):
        layers.append(causal_block(in_ch, hidden, dilation=2**i))
        in_ch = hidden
    self.tcn = nn.Sequential(*layers)

    self.head = nn.Sequential(
        nn.Linear(hidden + 2, 64),

```

```

        nn.ReLU(),
        nn.Linear(64, 1)
    )

    def forward(self, x_window, future_feats):
        x = x_window.permute(0, 2, 1)
        h = self.tcn(x)
        context = h[:, :, -1]
        B, H, _ = future_feats.shape
        context_rep = context.unsqueeze(1).expand(B, H, context.shape[1])
        z = torch.cat([context_rep, future_feats], dim=-1)

        out = self.head(z).squeeze(-1)
        return out

    def train_multi_tcn(resid_train: pd.Series, lookback=36, horizon=24,
epochs=800, batch_size=16, lr=1e-3):
        ds = ResidualMultiHorizonDataset(resid_train, lookback=lookback,
horizon=horizon)
        dl = DataLoader(ds, batch_size=batch_size, shuffle=True,
drop_last=True)

        model = MultiHorizonTCN(hidden=32, levels=4, kernel=3,
dropout=0.15, horizon=horizon).to(DEVICE)
        opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-4)
        loss_fn = nn.MSELoss()

        for ep in range(epochs):
            model.train()
            total = 0.0
            for xw, ff, yy in dl:
                xw = xw.to(DEVICE)
                ff = ff.to(DEVICE)
                yy = yy.to(DEVICE)

```

```

        pred = model(xw, ff)
        loss = loss_fn(pred, yy)

        opt.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step()
        total += float(loss.item())

    if (ep+1) % 200 == 0:
        print(f"Epoch {ep+1}/{epochs} loss={total/len(dl):.6f}")

    return model

@torch.no_grad()
def forecast_multi_tcn(model, resid_train: pd.Series, start_date,
horizon=24, lookback=36):
    model.eval()
    history = resid_train.values.astype(np.float32)
    xw = torch.tensor(history[-lookback:],
dtype=torch.float32).view(1, lookback, 1).to(DEVICE)

    idx = pd.date_range(start=pd.to_datetime(start_date),
periods=horizon, freq="MS")
    feats = []
    for d in idx:
        ms, mc = month_sin_cos(d.month)
        feats.append([ms, mc])
    feats = torch.tensor(np.array(feats,
dtype=np.float32)).unsqueeze(0).to(DEVICE)

    pred = model(xw, feats).cpu().numpy().flatten()
    return pd.Series(pred, index=idx)

```

A.6 RESULTS

```
def smape(y_true, y_pred):
    y_true = np.asarray(y_true); y_pred = np.asarray(y_pred)
    denom = (np.abs(y_true) + np.abs(y_pred)) + 1e-8
    return 100.0 * np.mean(2.0 * np.abs(y_pred - y_true) / denom)

def eval_metrics(y_true, y_pred, name):
    return {
        "Model": name,
        "MAE": float(mean_absolute_error(y_true, y_pred)),
        "RMSE": float(math.sqrt(mean_squared_error(y_true, y_pred))),
        "sMAPE": float(smape(y_true, y_pred)),
    }

y_true = test.values
y_bass = yhat_fore.loc[test.index].values
y_sar = y_fore_sarimax.loc[test.index].values
y_nn = y_fore_nn.loc[test.index].values
y_tcn = y_fore_tcn.loc[test.index].values
y_tcn_mh = y_fore_bass_tcn_mh.loc[test.index].values

results = pd.DataFrame([
    eval_metrics(y_true, y_bass, "Bass mean only"),
    eval_metrics(y_true, y_sar, "Bass + SARIMAX residual"),
    eval_metrics(y_true, y_nn, "Bass + NN residual (LSTM)",),
    eval_metrics(y_true, y_tcn, "Bass + NN residual (TCN)",),
    eval_metrics(y_true, y_tcn_mh, "Bass + Multi-horizon TCN residual")
])
print(results)
```

```

# RAW
y_true_raw = test_raw.values
y_bass_raw = yhat_fore_raw.loc[test_raw.index].values
y_sar_raw = y_fore_sarimax_raw.loc[test_raw.index].values
y_nn_raw = y_fore_nn_raw.loc[test_raw.index].values
y_tcn_raw = y_fore_tcn_raw.loc[test_raw.index].values
y_tcn_mh_raw = y_fore_bass_tcn_mh_raw.loc[test_raw.index].values

results_raw = pd.DataFrame([
    eval_metrics(y_true_raw, y_bass_raw, "Bass mean only - raw"),
    eval_metrics(y_true_raw, y_sar_raw, "Bass + SARIMAX residual - raw"),
    eval_metrics(y_true_raw, y_nn_raw, "Bass + NN residual (LSTM) - raw"),
    eval_metrics(y_true_raw, y_tcn_raw, "Bass + NN residual (TCN) - raw"),
    eval_metrics(y_true_raw, y_tcn_mh_raw, "Bass + Multi-horizon TCN
    residual - raw")
])
print(results_raw)

## Norway

y_true_2 = test_2.values
y_bass_2 = yhat_fore_2.loc[test_2.index].values
y_sar_2 = y_fore_sarimax_2.loc[test_2.index].values
y_nn_2 = y_fore_nn_2.loc[test_2.index].values
y_tcn_2 = y_fore_tcn_2.loc[test_2.index].values
y_tcn_mh_2 = y_fore_bass_tcn_mh_2.loc[test_2.index].values

results_2 = pd.DataFrame([
    eval_metrics(y_true_2, y_bass_2, "Bass mean only - Norway"),
    eval_metrics(y_true_2, y_sar_2, "Bass + SARIMAX residual - Norway"),
    eval_metrics(y_true_2, y_nn_2, "Bass + NN residual (LSTM) - Norway"),
    eval_metrics(y_true_2, y_tcn_2, "Bass + NN residual (TCN) - Norway"),
    eval_metrics(y_true_2, y_tcn_mh_2, "Bass + Multi-horizon TCN
    residual - Norway")
])

```



```

])
print(results_2)

# RAW Norway
y_true_raw_2 = test_raw_2.values
y_bass_raw_2 = yhat_fore_raw_2.loc[test_raw_2.index].values
y_sar_raw_2 = y_fore_sarimax_raw_2.loc[test_raw_2.index].values
y_nn_raw_2 = y_fore_nn_raw_2.loc[test_raw_2.index].values
y_tcn_raw_2 = y_fore_tcn_raw_2.loc[test_raw_2.index].values
y_tcn_mh_raw_2 = y_fore_bass_tcn_mh_raw_2.loc[test_raw_2.index].values

results_raw_2 = pd.DataFrame([
    eval_metrics(y_true_raw_2, y_bass_raw_2, "Bass mean only - raw - Norway"),
    eval_metrics(y_true_raw_2, y_sar_raw_2, "Bass + SARIMAX residual
- raw - Norway"),
    eval_metrics(y_true_raw_2, y_nn_raw_2, "Bass + NN residual
(LSTM) - raw - Norway"),
    eval_metrics(y_true_raw_2, y_tcn_raw_2, "Bass + NN residual (TCN)
- raw - Norway"),
    eval_metrics(y_true_raw_2, y_tcn_mh_raw_2, "Bass + Multi-horizon
TCN residual - raw - Norway")
])
print(results_raw_2)

```


References

- [1] S. Hardman, A. Chandan, G. Tal, and T. Turrentine, “The effectiveness of financial purchase incentives for battery electric vehicles – a review of the evidence,” *Renewable and Sustainable Energy Reviews*, vol. 80, pp. 1100–1111, 2017.
- [2] W. Sierzechula, S. Bakker, K. Maat, and B. van Wee, “The influence of financial incentives and other socio-economic factors on electric vehicle adoption,” *Energy Policy*, vol. 68, pp. 183–194, 2014.
- [3] E. M. Rogers, *Diffusion of Innovations*, 3rd ed. Free Press, 1983.
- [4] F. M. Bass, “A new product growth model for consumer durables,” *Management Science*, vol. 15, no. 5, pp. 215–227, 1969.
- [5] R. Peres, V. Mahajan, and E. Muller, “Innovation diffusion and new product growth models: A critical review and research directions,” *Journal of Research in Marketing*, vol. 27, no. 2, pp. 91–106, 2010.
- [6] V. Mahajan, E. Muller, and F. M. Bass, “New product diffusion models in marketing: A review and directions for research,” *Journal of Marketing*, vol. 54, no. 1, pp. 1–26, 1990.
- [7] M. Guidolin, *Innovation Diffusion Models: Theory and Practice*. Wiley, 2023.
- [8] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>
- [9] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. John Wiley & Sons, 2015.
- [10] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [11] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” *arXiv preprint arXiv:1803.01271*, 2018.

- [12] R. Guseo and M. Guidolin. (2009) Market potential dynamics in innovation diffusion: Modelling the synergy between two driving forces. [Online]. Available: https://www.research.unipd.it/retrieve/e14fb270-2c1a-3de1-e053-1705fe0ac030/2009_10_20090525131912.pdf
- [13] F. Sprei, “Disrupting mobility,” *Energy Research and Social Science*, vol. 37, pp. 238–242, 2018.
- [14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, <http://www.deeplearningbook.org>.
- [15] Norwegian Public Roads Administration, “Vehicle registration data,” <https://hotell.difi.no/?dataset=vegvesen/kjoretoyinfo2>, 2026, (accessed: 20.03.2026).
- [16] UNRAE, “Vehicle registration statistics: Market structure,” <https://unrae.it/dati-statistici/immatricolazioni/2681/struttura-del-mercato-gennaio-2014>, 2026, (accessed: 20.03.2026).

Acknowledgments

I would like to express my sincere gratitude to prof. Guidolin for her guidance, valuable feedback, and continuous support throughout the development of this research.

I would also like to thank the University of Padova for providing an enriching academic environment that allowed me to deepen my knowledge and develop the analytical skills necessary to carry out this work.

Finally, I would like to express my deepest appreciation to my family and friends for their constant support, patience, and encouragement during this journey. Their motivation and belief in me made the completion of this thesis possible.